

CS570
Analysis of Algorithms
Fall 2006
Exam 1

Name: _____

Student ID: _____

	Maximum	Received
Problem 1	20	
Problem 2	10	
Problem 3	10	
Problem 4	10	
Problem 5	10	
Problem 6	20	
Problem 7	20	

Note: The exam is closed book closed notes.

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[**TRUE/FALSE**] True (By definition it is true)

If $T(n)$ is both $O(f(n))$ and $\Omega(f(n))$, then $T(n)$ is $\Theta(f(n))$.

[**TRUE/FALSE**] True

For a graph G and a node v in that graph, the DFS and BFS trees of G rooted at v always contain the same number of edges

[**TRUE/FALSE**] False (Counter example: Fibonacci Heap)

Complexity of the "Decrease_Key" operation is always $O(\lg n)$ for a priority queue.

[**TRUE/FALSE**] True (See the solution of HW4)

For a graph with distinct edge weights there is a unique MST.

[**TRUE/FALSE**] True (yes and store all the possible solution in a table)

Dynamic programming considers all the possible solutions.

[**TRUE/FALSE**] False (The graph a-b-c-d-a with three edges are 1 and the one left is 4)

Consider an undirected graph $G=(V, E)$ and a shortest path P from s to t in G . Suppose we add one 1 to the cost of each edge in G . P will still remain as a shortest path from s to t .

[**TRUE/FALSE**] True

Consider an undirected graph $G=(V, E)$ and its minimum spanning tree T . Suppose we add one 1 to the cost of each edge in G . T will still remain as an MST.

[**TRUE/FALSE**] False (Counter example: Shortest Path in a Graph)

Problems solved using dynamic programming cannot be solved thru greedy algorithms.

[**TRUE/FALSE**] False (union-Find data structure is for Kruskal's algorithm, it is not for the reverse delete. Check the textbook for more details)

The union-Find data structure can be used for an efficient implementation of the reverse delete algorithm to find an MST.

[**TRUE/FALSE**] False (Prim algorithm Vs Kruskal algorithm, return can be different)

While there are different algorithms to find a minimum spanning tree of undirected connected weighted graph G , all of these algorithms produce the same result for a given G .

2) 10 pts

Indicate for each pair of expressions (A,B) in the table below, whether A is **O**, **Ω** , or **Θ** of B. Assume that k and c are positive constants. You can mark each box with Y (yes) and N (no).

A	B	O	Ω	Θ
$n^3 + n^2 + n + c$	n^3	Yes	Yes	Yes
2^n	$2^{(n+k)}$	Yes	Yes	Yes
n^2	$n \cdot 2^{\log(n)}$	No	Yes	No

3) 10 pts

a- What is the minimum and maximum numbers of elements in a heap of height h ?

The minimum is $2^{(h-1)}$

The maximum number is $2^h - 1$

b- What is the number of leaves in a heap of size n ?

The smallest integer that no less than $n/2$.

Or

When n is even, the number of leaves is $n/2$;

When n is odd; the number of leaves is $(n+1)/2$

c- Is the sequence $\langle 23, 7, 14, 6, 13, 10, 1, 5, 17, 12 \rangle$ a max-heap? If not, show how to heapify the sequence.

Answer: No. The sequence in heapify is

$\langle 23, 7, 14, 17, 13, 10, 1, 5, 6, 12 \rangle$, $\langle 23, 17, 14, 7, 13, 10, 1, 5, 6, 12 \rangle$

d- Where in a max-heap might the smallest element reside, assuming that all elements are distinct.

The smallest element might reside in any leaves

Grading policy: 2 points for a); 2 points for b); 3 points for c); 3 points for d)

4) 10 pts

Prove or disprove the following:

The shortest path between any two nodes in the minimum spanning tree $T = (V, E')$ of connected weighted undirected graph $G = (V, E)$ is a shortest path between the same two nodes in G . Assume the weights of all edges in G are unique and larger than zero.

False. Consider the graph A-B-C-D-A with weight 1,2,3,4

Grading policy:

- Any one says that the statement is true and try to prove it (of course with wrong proof) may get partial credit up to 3 marks.

- Any one says it is false without correct disproof (counter example) may get partial credit up to 6 marks.

- In Question 5, any one says it is false and give a counter example that has one or more nodes in both G_1 and G_2 may get partial credit up to 6 marks. Because that is a mistake, nodes in G_1 can not exist in G_2 and vice versa.

- Any one says it is false and give correct disprove (counter example) get 10 out of 10.

5) 10 pts

Suppose that you divided a graph $G = (V, E)$ into two sub graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$. And, we can find M_1 which is a MST of G_1 and M_2 which is MST of G_2 . Then, $M_1 \cup M_2 \cup \{\text{minimum weight edge among those connecting two graph } G_1 \text{ and } G_2\}$ always gives MST of G . Prove it or disprove it.

Solution: False.

Counter example: Consider a graph G composed of 4 nodes: A,B,C,D with edge $w(A,B)=1, w(B,C)=1, w(B,D)=1, w(C,D)=2$. Let $V_1=\{A,B\}$ with edge AB, $V_2=\{C,D\}$ with edge CD. Then the weight of $M_1 \cup M_2 \cup$ is 4 while the weight of the MST of G is 3.

Grading policy:

- Any one says that the statement is true and try to prove it (of course with wrong proof) may get partial credit up to 3 marks.

- Any one says it is false without correct disproof (counter example) may get partial credit up to 6 marks.

- In Question 5, any one says it is false and give a counter example that has one or more nodes in both G_1 and G_2 may get partial credit up to 6 marks. Because that is a mistake, nodes in G_1 can not exist in G_2 and vice versa.

- Any one says it is false and give correct disprove (counter example) get 10 out of 10.

6) 20 pts

There are n workers in the factory with heights of $p_1, p_2, \dots, p_n$, and n working-clothes with height sizes of $s_1, s_2, \dots, s_n$. The problem is to find best matching strategy such that we minimize the following average differences.

$$\frac{1}{n} \sum |p_i - s_i|$$

Present an algorithm to solve this problem along with its proof of correctness.

Solution: Sort the height of workers in increasing order $p_1 \leq p_2 \leq \dots \leq p_n$

Sort the height size of clothes in increasing order $s_1 \leq s_2 \leq \dots \leq s_n$

Match h_i with s_i is the best matching strategy such that we minimize the following average differences.

$$\frac{1}{n} \sum |p_i - s_i|$$

The algorithm is correct for the problem of minimizing the average difference between the heights of workers and their clothes. The proof is by contradiction. Assume the people and clothes are numbered in increasing order by height. If the greedy algorithm is not optimal, then there is some input $p_1, \dots, p_n, s_1, \dots, s_n$ for which it does not

produce an optimal solution. Let the optimal solution be $T = \{(p_1, s_1), \dots, (p_n, s_n)\}$, and let the output of the greedy algorithm be $G = \{(p_1, s_1), \dots, (p_n, s_n)\}$. Beginning with p_1 , compare T and G . Let p_i be the first person who is assigned different cloth in G than in T . Let s_j be the pair of cloth assigned to p_i in T . Create solution T' by switching the cloth assignments of p_i and p_j . By the definition of the greedy algorithm, s_i is equal or greater than s_j . The total cost of T' is given by

$$Cost(T') = Cost(T) - \frac{1}{n}(|p_i - s_j| + |p_j - s_i| - |p_i - s_i| - |p_j - s_j|)$$

There are six cases to be considered. For each case, one needs to show that $(|p_i - s_j| + |p_j - s_i| - |p_i - s_i| - |p_j - s_j|) \geq 0$.

Case 1: $p_i \leq p_j \leq s_i \leq s_j$.

$$\begin{aligned} |p_i - s_j| + |p_j - s_i| - |p_i - s_i| - |p_j - s_j| &= \\ (s_j - p_i) + (s_i - p_j) - (s_i - p_i) - (s_j - p_j) &= 0 \end{aligned}$$

Case 2: $p_i \leq s_i \leq p_j \leq s_j$.

$$\begin{aligned} |p_i - s_j| + |p_j - s_i| - |p_i - s_i| - |p_j - s_j| &= \\ (s_j - p_i) + (p_j - s_i) - (s_i - p_i) - (s_j - p_j) &= \\ 2(p_j - s_i) &\geq 0 \end{aligned}$$

Case 3: $p_i \leq s_i \leq s_j \leq p_j$.

$$\begin{aligned} |p_i - s_j| + |p_j - s_i| - |p_i - s_i| - |p_j - s_j| &= \\ (s_j - p_i) + (p_j - s_i) - (s_i - p_i) - (p_j - s_j) &= \\ 2(s_j - s_i) &\geq 0 \end{aligned}$$

Case 4: $s_i \leq s_j \leq p_i \leq p_j$.

$$\begin{aligned} |p_i - s_j| + |p_j - s_i| - |p_i - s_i| - |p_j - s_j| &= \\ (p_i - s_j) + (p_j - s_i) - (p_i - s_i) - (p_j - s_j) &= 0 \end{aligned}$$

Case 5: $s_i \leq p_i \leq s_j \leq p_j$.

$$\begin{aligned} |p_i - s_j| + |p_j - s_i| - |p_i - s_i| - |p_j - s_j| &= \\ (s_j - p_i) + (p_j - s_i) - (p_i - s_i) - (p_j - s_j) &= \\ 2(s_j - p_i) &\geq 0 \end{aligned}$$

Case 6: $s_i \leq p_i \leq p_j \leq s_j$.

$$\begin{aligned} |p_i - s_j| + |p_j - s_i| - |p_i - s_i| - |p_j - s_j| &= \\ (s_j - p_i) + (p_j - s_i) - (p_i - s_i) - (s_j - p_j) &= \\ 2(p_j - p_i) &\geq 0 \end{aligned}$$

Running time: Sorting takes $O(n \log n)$ and hence the running time is $O(n \log n)$

Grading policy: Correct description of algorithm 10 pts

Based on the above correct algorithm, present correct complexity 4pts

Correct proof (all six cases) 6pts

7) 20 pts

Given an unlimited supply of coins of denominations $x_1, x_2, \dots, x_n$, we wish to make change for a value v , that is, we wish to find a set of coins whose total value is v . This might not be possible: for example, if the denominations are 5 and 10 then we can make change for 15 but not for 12. Give an $O(nv)$ algorithm to determine if it is possible to make change for v using coins of denominations $x_1, x_2, \dots, x_n$.

Solution: We solve this question by dynamic programming. Denote $OPT(i)$ as the possibility of paying value i using coins of denominations $x_1, x_2, \dots, x_n$. Then $OPT(i)$ is true if and only if we can pay $i - x_1$, or $i - x_2$, ... or $i - x_n$ using coins of denominations $x_1, x_2, \dots, x_n$. In other words,

$OPT(i) = OPT(i - x_1) \vee OPT(i - x_2) \vee \dots \vee OPT(i - x_n)$

Following the recursive relation with initial value $OPT(x_1) = \dots = OPT(x_n) = \text{true}$ we compute the value of $OPT(v)$. If $OPT(v)$ is true, then we can change for v using coins of denominations $x_1, x_2, \dots, x_n$, otherwise we can not.

Clearly to compute $OPT(v)$ we need an array of size v , and each time when we compute $OPT(i)$, we do n union operations. Hence the running time is $O(nv)$

The following pseudo-code would help you understand the solution:

Data structures: $A[v, n]$: 2-dimensional array with entries T or F;

$OPT[1..n]$: 1-dimensional array with entries T or F.

```
for (j=1 to v)
  OPT(j) = F;
for (j=1 to v)
  for (i=1 to n)
  {
    if (j < xi) then A[j,i] = F;
    if (j == xi) then A[j,i] = T;
    if (j > xi) then A[j,i] = OPT(xi) AND OPT(j-xi);
    if (A[j,i] == T) then OPT(j) = T;
  }
return OPT(v)
```


Additional Space

Additional Space

CS570
Analysis of Algorithms
Fall 2008
Exam I

Name: _____
Student ID: _____

____Monday Section ____Wednesday Section ____Friday Section

	Maximum	Received
Problem 1	20	
Problem 2	10	
Problem 3	10	
Problem 4	20	
Problem 5	20	
Problem 6	20	
Total	100	

2 hr exam
Close book and notes

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[**TRUE/FALSE**] F

Given graph G and a Minimum Spanning Tree T on G , you could find the (weighted) shortest path between arbitrary pair u, v in $V(G)$ using only edges in T .

[**TRUE/FALSE**] F

$V^2 \log V = \theta(E \log E^2)$ whether the graph is dense or sparse.

[**TRUE/FALSE**] T

If DFS and BFS returns different trees, then the original graph is not a tree.

[**TRUE/FALSE**] T

An algorithm with the running time of $n * 2^{\min(n \log n, 10000)}$ runs in polynomial time.

[**TRUE/FALSE**] T

We may need to run Dijkstra's algorithm to compute the shortest path on a directed graph, even if the graph doesn't have a cycle.

[**TRUE/FALSE**] F

Given a graph that contains negative edge weights, we can use Dijkstra's algorithm to find the shortest paths between any two vertexes by first adding a constant weight to all of the edges to eliminate the negative weights.

[**TRUE/FALSE**] F

If $f(n)$ and $g(n)$ are asymptotically positive functions, then $f(n)+g(n) = \theta(\min\{f(n), g(n)\})$.

[**TRUE/FALSE**] F

For a stable matching problem such that m ranks w last and w ranks m last, m and w will never be paired in a stable matching.

[**TRUE/FALSE**] F

Breadth first search is an example of a divide-and-conquer algorithm.

[**TRUE/FALSE**] T

Kruskal's algorithm for finding the MST works with positive and negative edge weights.

2) 10 pts

a) Arrange the following in the increasing order of asymptotic growth. Identify any ties.

$\lg n^{10}, 3^n, \lg n^{2n}, 3n^2, \lg n^{\lg n}, 10^{\lg n}, n^{\lg n}, n \lg n$

$\lg n^{10}, \lg n^{\lg n}, \lg n^{2n}, n \lg n, 3n^2, 10^{\lg n}, n^{\lg n}, 3^n$

b) Analyze the complexity of the following loops:

```
i- x = 0
   for i=1 to n
       x = x + lg n
   end for
```

$O(n)$

```
ii- x=0
    for i=1 to n
        for j=1 to lg n
            x = x * lg n
        endfor
    endfor
```

$O(n \lg n)$

```
iii- x = 0
     k = "some constant"
     for i=1 to max (n, k)
         x = x + lg n
     end for
```

$O(n)$

```
iv- x=0
    k = "some constant"
    for i=1 to min(n, k)
        for j=1 to lg n
            x = x * lg n
        endfor
    endfor
```

$O(\lg n)$

3) 10 pts

a) For each of the following recurrences, give an expression for the runtime $T(n)$ if the recurrence can be solved with the Master Theorem. Otherwise, indicate that the Master Theorem does not apply.

$$T(n) = T(n/2) + 2^n$$
$$\theta(2^n)$$

$$T(n) = 16T(n/4) + n$$
$$\theta(n^2)$$

$$T(n) = 2T(n/2) + n \log n$$

Master Theorem does not apply.

$$T(n) = 2T(n/2) + \log n$$
$$\theta(n)$$

$$T(n) = 64T(n/8) - n^2 \log n$$

Master Theorem does not apply.

b) Suppose we have a divide and conquer algorithm which at each step takes time n , and breaks the problem up into $n^{1/2}$ subproblems of size $n^{1/2}$ each. Find the runtime $f(n)$ such that $T(n) = \theta(f(n))$, given the following recurrence. $T(n) = n^{1/2} T(n^{1/2}) + n$

They have to show the full recursion tree.

At the root we spend cn

At the next level we have $n^{1/2}$ nodes each taking $cn^{1/2}$ so again, at this level we spend cn time

Same goes with every other level. There are $\lg n$ levels in the tree because at every we take the square root of n . So the total run time is $\theta(n \lg n)$

4) 20 pts

A key to customer service is to avoid having the customer wait longer than necessary. That's your philosophy anyway when you open a coffee shop shortly after graduation. This philosophy and your CS background have made you wonder about the order in which your server should serve customers whose orders have different levels of complexity. For example, if customer one's order will take 3 minutes while customer two's will take 1 minute, then serving customer one first results in 7 minutes of waiting (3 for customer one and 4 for customer two) while serving customer two first results in only 5 (1 for customer two and 4 for customer one).

(a) Phrase this problem more formally by introducing notation for the service time and precisely define the problem's objective.

[Please refer to solution to Q4](#)

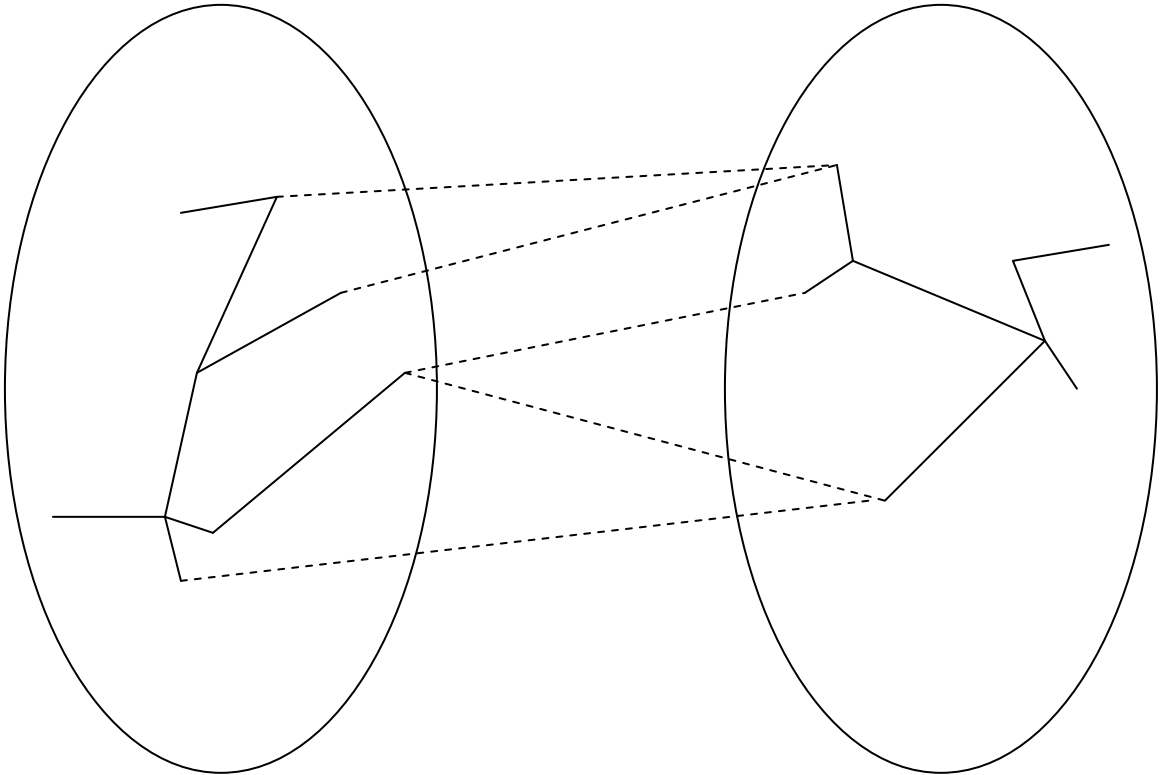
(b) Give a greedy algorithm and use an interchange argument to prove that it works for a single server as long as no new customers arrive.

(c) Show how your algorithm can fail if new customers arrive while the others are being served. (For this problem, assume that you cannot stop serving a customer once you start; coffee equipment is not designed for context switching.)

(d) Show that your algorithm does not necessarily find the best solution if the coffee shop has two servers.

5) 20 pts

Assume we have two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$. Also assume that we have T_1 which is a MST of G_1 and T_2 which is MST of G_2 . Now consider a new graph $G = (V, E)$ such that $V = V_1 \cup V_2$ and $E = E_1 \cup E_2 \cup E_3$ where E_3 is a new set of edges that all cross the cut (V_1, V_2) . Following is an example of what G might look like.



The dashed edges are E_3 , the solid edges in G_1 (on the left) are T_1 , and the solid edges in G_2 (on the right) are T_2 .

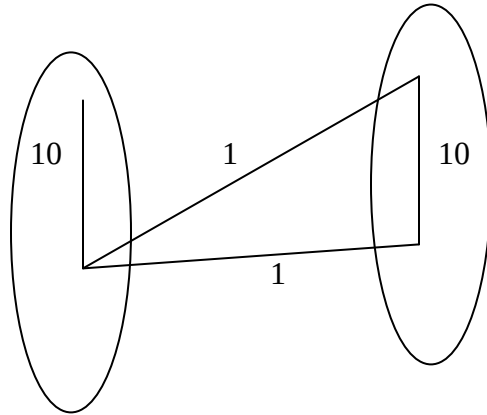
Now assume we want to find a MST of the new graph. Consider the following algorithm:

```
Maybe-MST( $T_1, T_2, E_3$ )  
 $e_{\min}$  = a minimum weight edge in  $E_3$   
 $T = T_1 \cup T_2 \cup \{e_{\min}\}$   
return  $T$ 
```

Prove or disprove this algorithm (space provided on next page)

Additional space

Can show a simple counter example.



6) 20 pts

For an unsorted array x of size n , give a divide and conquer algorithm that runs in $O(n)$ time and computes the maximum sum M of two adjacent elements in an array.

$$M = \text{Max } (x_i + x_{i+1}); \quad i=1 \text{ to } n-1$$

Divide: divide problem into 2 equal size subproblems at each step

Conquer: solve the subproblems recursively until the subproblem become trivial to solve. In this case problem size of 2 is trivial. In that case the solution is the sum of the 2 numbers.

Combine: We need to merge the solutions to the two subproblems S1 and S2. At each step we need to evaluate the following 3 cases:

Case 1: Max is in S1 (subproblem to the left)

Case 2: Max is in S2 (subproblem to the right)

Cases 3: Max is on the border of S1 and S2

To achieve this. In addition to finding the Max and sending it up the recursion tree at each step we need to send the two numbers at the ends of the subarrays S1 and S2 at each step. So using the following convention:

F1 = first element of S1

L1 = last element of S1

MAX1 = maximum sum of two adjacent elements in S1

F2 = first element of S2

L2 = last element of S2

MAX2 = maximum sum of two adjacent elements in S2

Then we will combine the two subproblems as follows:

If ($L1+F2 > \text{MAX1}$ and $L1+F2 > \text{MAX2}$) then

$F = F1$

$L = L2$

$\text{MAX} = L1+F2$

Else if ($\text{MAX1} > \text{MAX2}$) then

$F = F1$

$L = L2$

$\text{MAX} = \text{MAX1}$

Else

$F = F1$

$L = L2$

$\text{MAX} = \text{MAX2}$

endif

Additional Space

Additional Space

x

CS570
Analysis of Algorithms
Fall 2009
Exam I

Name: _____

Student ID: _____

____Monday ____Friday 2-5 ____Friday 5-8 ____DEN

	Maximum	Received
Problem 1	20	
Problem 2	15	
Problem 3	15	
Problem 4	15	
Problem 5	15	
Problem 6	20	
Total	100	

2 hr exam
Close book and notes

If a description of an algorithm is required, please limit your description within 200 words, anything beyond 200 words will not be considered. Proof/justification or complexity analysis will only be considered if the algorithm is either correct or provided by the problem.

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[TRUE]

Given a min-heap with n elements, the time complexity of select the smallest element from the n elements is $O(1)$.

[FALSE] But we give credit to both true and false

Given a min-heap with n elements, the time complexity of select the second smallest element from the n elements is $O(1)$.

[FALSE] Algorithms (e.g., bubble sort) with $O(n^2)$ time complexity could cost $O(n)$ in some instances.

Given a problem with input of size n , a solution with $O(n)$ time complexity always costs less in computing time than a solution with $O(n^2)$ time complexity.

[FALSE]

By using a heap, we can sort any array with n integers in $O(n)$ time.

[TRUE] m is at most $n(n-1)$

For any graph with n vertices and m edges we can say that $m = O(n^2)$.

[FALSE] m could be $n(n-1)$

For any graph with n vertices and m edges we can say that $O(m+n) = O(m)$.

[FALSE] Consider the following counter-example:

$G(V, E)$. $V=\{A,B,C\}$, $(A,B)=2$, $(A,C)=1$, $(B,C)=1$. $P=AB$ is the shortest path between A and B, but it is not in the MST.

Given the shortest path P between two nodes A and B in graph $G(V,E)$, then there exists a minimum spanning tree T of G , such that all edges of path P is contained in T .

[FALSE] Consider the following counter-example:

w_1 prefers m_1 to m_2 ; w_2 prefers m_2 to m_1 ; m_1 prefer w_1 to w_2 ; m_2 prefer w_2 to w_1

Consider an instance of the Stable Matching Problem in which there exists a man m and a woman w such that m is ranked last on the preference list of w and w is ranked last on the preference list of m , then in every stable matching S for this instance, the pair (w, m) belongs to S .

[FALSE] A graph could have multiple MSTs.

While there are many algorithms to find the minimum spanning tree in a graph, they all produce the same minimum spanning tree.

[TRUE]

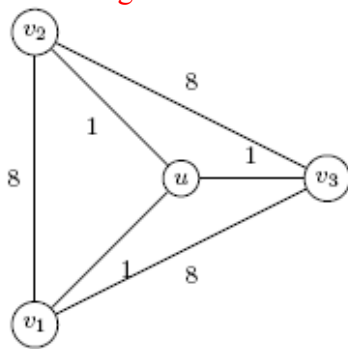
A BFS tree is a spanning tree

2) 15 pts

Here is a divide-and-conquer algorithm that aims at finding a minimum spanning tree. Given a graph $G = (V, E)$, partition the set V of vertices into two sets V_1 and V_2 such that $|V_1|$ and $|V_2|$ differ by at most 1. Let E_1 be the set of edges that are incident only on vertices in V_1 , and let E_2 be the set of edges that are incident only on vertices in V_2 . Recursively solve a minimum spanning tree problem on each of the two subgraphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$. Finally, select the minimum-weight edge in E that crosses the cut (V_1, V_2) , and use this edge to unite the resulting two minimum spanning trees into a single spanning tree.

Either argue that the algorithm correctly computes a minimum spanning tree of G , or provide an example for which the algorithm fails.

This algorithm fails. Take the following simple graph:



Never mind how the algorithm first divides the graph, the MST in one subgraph (the one containing u) will be one edge of cost 1, and in the other subgraph, it will be one edge of cost 8. Adding another edge of cost 1 will give a tree of cost 10. But clearly, it would be better to connect everyone to u via edges of cost 1.

3) 15 pts

Suppose you are given two sets A and B , each containing n positive integers. You can choose to reorder each set however you like. After reordering, let a_i be the i th element of set A , and let b_i be the i th element of set B . You then receive a payoff of $\prod_{i=1}^n a_i^{b_i}$. Give an algorithm that will maximize your payoff. Prove that your algorithm maximizes the payoff, and state its running time.

Solution:

Algorithm:

Sort A and B into monotonically decreasing order.

Proof:

Consider any indices i, j, p, q such that $i < j$ and $p < q$, and consider the terms $a_i^{b_p}$ and $a_j^{b_q}$. We want to show that it is no worse to include these terms in the payoff than to include $a_i^{b_q}$ and $a_j^{b_p}$, that is, $a_i^{b_p} a_j^{b_q} \geq a_i^{b_q} a_j^{b_p}$.

Since A and B are sorted into monotonically decreasing order and $i < j, p < q$, we have $a_i \geq a_j$ and $b_p \geq b_q$. Since a_i and a_j are positive and $b_p - b_q$ is nonnegative, we have $a_i^{b_p - b_q} \geq a_j^{b_p - b_q}$.

Multiplying both sides by $a_i^{b_q} a_j^{b_q}$ yields $a_i^{b_p} a_j^{b_q} \geq a_i^{b_q} a_j^{b_p}$.

Since the order of multiplication does not matter, sorting A and B into monotonically increasing order works as well.

Complexity:

Sorting 2 sets of n positive integers is $O(n \log n)$.

4) 15 pts

Indicate, for each pair of expressions (A, B) in the table below, whether A is O , Ω or Θ of B. Assume that $k \geq 1$, and $\varepsilon > 0$ are constants. Answer “yes” or “no” in each box in the following form. No explanations required.

A	B	O	Ω	Θ
$\log^k n$	n^ε	Yes	No	No
$\sqrt{n}$	$n^{\sin n}$	No	No	No
$n^{\log m}$	$m^{\log n}$	Yes	Yes	Yes

The explanation is not required:

- (1) $\log(\log n) = O(\log n) \Rightarrow k \log(\log n) = O(\varepsilon \log n) \Rightarrow \log(\log^k n) = O(\log n^\varepsilon) \Rightarrow \log^k n = O(n^\varepsilon)$
- (2) $-1 \leq \sin n \leq 1$, so we can not determine which one is larger;
- (3) $n^{\log m} = \Theta(n^{\lg m})$, $m^{\log n} = \Theta(m^{\lg n})$, let $m = 10^x$, and $n = 10^y$, then $n^{\lg m} = m^{\lg n} = 2^{xy}$.

5) 15 pts

Suppose you are given a number x and an array A with n entries, each being a distinct number. Also it is known that the sequence of values $A[1]; A[2]; \dots; A[n]$ is unimodal.

In other words for some unknown index p between 1 and n , we have

$A[1] < A[2] < \dots < A[p]$ and $A[p] > A[p+1] > \dots > A[n]$.

Give an algorithm with running time $O(\log n)$ to find out if x belongs to A , if yes the algorithm should return the index j such that $A[j] = x$. You should justify both your algorithm and the running time.

Solution: The idea is to first find out p and then break A into two separated sorted arrays, then use binary search on these two arrays to check if x belongs to A .

Let $\text{FindPeak}()$ be the function of finding the peak in A . Then $\text{FindPeak}(A[1 : n])$ works as follows:

Look at $A[n/2]$, there are 3 cases:

(1) If $A[n/2 - 1] < A[n/2] < A[n/2 + 1]$, then the peak must come strictly after $n/2$.

Run $\text{FindPeak}(A[n/2 : n])$.

(2) If $A[n/2 - 1] > A[n/2] > A[n/2 + 1]$, then the peak must come strictly before $n/2$.

Run $\text{FindPeak}(A[1 : n/2])$.

(3) If $A[n/2 - 1] < A[n/2] > A[n/2 + 1]$, then the peak is $A[n/2]$, return $n/2$.

Now we know the peak index (p value). Then we can run binary search on $A[1 : p]$ and $A[p + 1 : n]$ to see if x belongs to them because they are both sorted.

In the procedure of finding p , we halve the size of the array in each recurrence. The running time $T(n)$ satisfies $T(n) = T(n/2) + O(1)$. Thus $T(n) = O(\log n)$. Also both binary search has running time at most $O(\log n)$, so total running time is $O(\log n)$.

Explanations:

Note that trying to get x in one shot (in one divide and conquer recursion) usually does not work. Binary search certainly cannot work because the array is not sorted. Modified binary search can hardly work either. The problem is that in each round you need to abandon half the array. However, this decision is hard to make. For example, the array is $[1, 2, 3, 4, 5, -1]$, $x = -1$. When divide we have $\text{mid} = 3$. We find $A[\text{mid} - 1] < A[\text{mid}] < A[\text{mid} + 1]$. A common but wrong practice here is to continue search only in the $A[1 : \text{mid}]$. We can see clearly $x = -1$ is in the second half of the array. On the other hand you cannot throw away the first half of the array either. The counter example is $[1, 2, 3, 4, 5, -1]$ where $x = 1$.

One other mistake is to search p in a linear fashion. This takes $O(n)$ in the worst case.

6) 20 pts

Given a string S with m digits (digits are from 1 to 9), you are required to delete n ($n < m$) digits from S . After the operation, you have a string S' with $m-n$ digits, let N denote the number that S' represents. Design a greedy algorithm to minimize N . For example, $S = "456298111"$, $n = 3$, after deleting 4, 5 and 6, you get the minimal $N = 298111$. Prove the correctness of your algorithm and analyze its time complexity.

Solution:

The greedy strategy: every step we delete a digit, make sure the number represented by the rest of the digits is minimal. To achieve this, in each step, we do the following operation:

Find the first i such that $S[i] > S[i+1]$, then we delete $S[i]$; if such i does not exist, which means the digits are in increasing order, then we simply delete the last digit. (*)

We call the position i "peak".

Proof by induction

(1) We first prove (*) can achieve minimal N when $n = 1$. We have

$$S' = a_1 a_2 \dots a_{i-1} a_{i+1} \dots a_{n-1}$$

Without loss of generality, suppose we delete $S[j]$ rather than $S[i]$, then we have

$$S_1 = a_1 a_2 \dots a_{j-1} a_{j+1} \dots a_{n-1} \quad (j \neq i), \text{ and } N_1 \text{ is the number represented by } S_1.$$

If $j < i$, since $a_j < a_{j+1}$, we have $N < N_1$; if $j > i$, since $a_{i+1} < a_i$, we have $N < N_1$.

Therefore, $N < N_1$ for all $j \neq i$.

(2) Next, we assume (*) can achieve minimal N when $n=k$, now we prove (*) can achieve minimal N when $n=k+1$.

Suppose the first deletion, we delete p' , then for the rest of k deletions, we need to do (*) k times (deleting the first k peaks) in order to get the minimal result, which is denoted by N' ;

Let N denote the number generated by deleting the first $k+1$ peaks, $p_1, p_2, \dots, p_{k+1}$, in S .

If $p' = p_i$ ($1 \leq i \leq k+1$), then $N' = N$; if $p' < p_1$ or $p_i < p' < p_{i+1}$ or $p' > p_{k+1}$, similar to (1), we can prove $N' > N$; Therefore, we show that $N \leq N'$.

(3) With (1) and (2), we prove the correctness of the algorithm.

Complexity:

$O(n)$, but $O(mn)$ is also acceptable.

CS570
Analysis of Algorithms
Fall 2010
Exam I

Name: _____

Student ID: _____

____Monday ____Friday ____DEN

	Maximum	Received
Problem 1	20	
Problem 2	15	
Problem 3	15	
Problem 4	15	
Problem 5	15	
Problem 6	20	
Total	100	

2 hr exam

Close book and notes

If a description to an algorithm is required please limit your description to within 150 words, anything beyond 150 words will not be considered.

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[**TRUE**]

The number of spanning trees in a fully connected graph with n vertices goes up exponentially with respect to n .

[**FALSE**]

BFS can be used to find the shortest path between any two nodes in a weighted graph.

[**FALSE**]

DFS can be used to find the shortest path between any two nodes in a non-weighted graph.

[**TRUE**]

While there are different algorithms to find a minimum spanning tree of an undirected connected weighted graph G , all of these algorithms produce the same result for a given graph with unique edge costs.

[**TRUE**]

If $T(n)$ is $\Theta(f(n))$, then $T(n)$ is both $O(f(n))$ and $\Omega(f(n))$.

[**TRUE**]

The array [20 15 18 7 9 5 12 3 6 2] forms a max-heap.

[**TRUE**]

Suppose that in an instance of the original Stable Marriage problem with n couples, there is a man M who is first on every woman's list and a woman W who is first on every man's list. If the Gale-Shapley algorithm is run on this instance, then M and W will be paired with each other.

[**TRUE**]

The complexity of the recursion given by $T(n) = 4T(n/2) + cn^2$, for some positive constant c , is $O(n^2 \log n)$.

[**FALSE**]

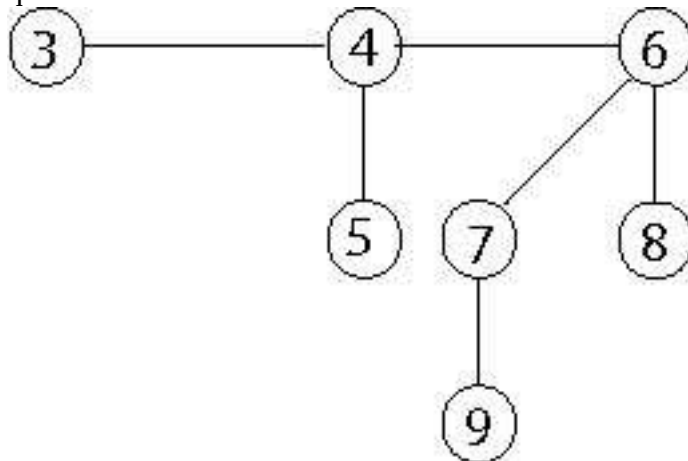
Consider the interval scheduling problem. A greedy algorithm, which is designed to always select the available request that starts the earliest, returns an optimal set A .

[**FALSE**]

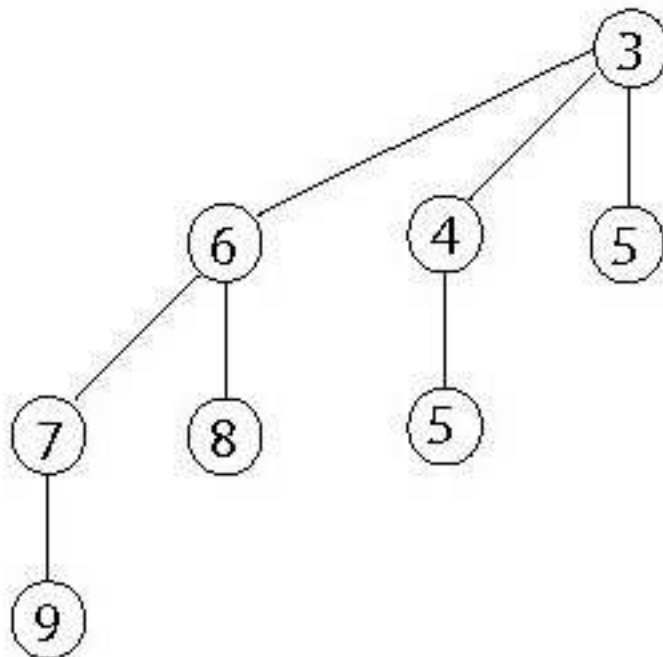
Any divide and conquer algorithm will run in best case $\Omega(n \log n)$ time because the height of the recursion tree is at least $\log n$.

2) 15 pts

You are given the below binomial heap. Show all your work as you answer the questions below.



a- Insert a new node with key value 5. Show the resulting tree and intermediate steps if any. Is the resulting heap also a binary heap and/or a Fibonacci heap?



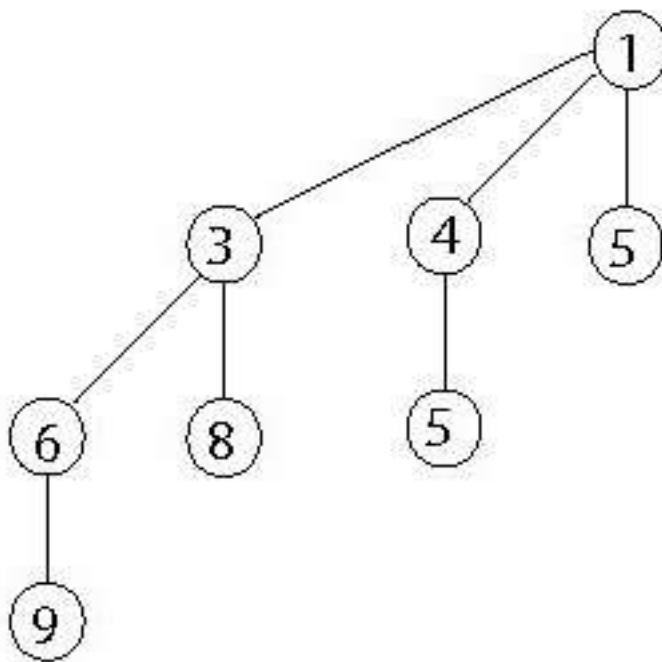
This is also a Fibonacci heap, but not binary heap.

b- Analyze the complexity of your insertion algorithm.

$O(\log n)$

c- Now decrease the key of node 7 to 1. Is the minimum-heap property violated? If so, rearrange the heap. Show the resulting tree and intermediate steps if any.

When key value is changed to 1, min-heap property is violated since 1 is the smallest key value and it has to be at the root node. The rearrangement of key values results in the following heap.



d- Analyze the complexity of the operation in C.

$O(\log n)$

3) 15 pts

A polygon is convex if all of its internal angles are less than 180° (and none of the edges cross each other). We represent a convex polygon as an array $V[1..n]$ where each element of the array represents a vertex of the polygon in the form of a coordinate pair (x, y) . We are told that $V[1]$ is the vertex with the minimum x coordinate and that the vertices $V[1..n]$ are ordered counterclockwise. You may also assume that the x coordinates of the vertices are all distinct, as are the y coordinates of the vertices.

a- Give a divide and conquer algorithm to find the vertex with the maximum x coordinate in $O(\log n)$ time.

Note that for each $1 \leq i < n$ either $V[i] < V[i + 1]$ or $V[i] > V[i + 1]$ (Such an array is called a unimodal array). The main idea is to distinguish these two cases:

1. if $V[i] < V[i + 1]$, then the maximum element of $V[1..n]$ occurs in $V[i + 1..n]$.
 2. In a similar way, if $V[i] > V[i + 1]$, then the maximum element of $V[1..n]$ occurs in $V[1..i]$.
- This leads to the following divide and conquer solution (note its resemblance to binary search):

```
1  $a, b \leftarrow 1, n$ 
2 while  $a < b$ 
3   do  $mid \leftarrow \lfloor (a + b)/2 \rfloor$ 
4     if  $V[mid] < V[mid + 1]$ 
5       then  $a \leftarrow mid + 1$ 
6     if  $V[mid] > V[mid + 1]$ 
7       then  $b \leftarrow mid$ 
8 return  $V[a]$ 
```

The precondition is that we are given a unimodal array $V[1..n]$. The postcondition is that $V[a]$ is the maximum element of $V[1..n]$. For the loop we propose the invariant "The maximum element of $V[1..n]$ is in $V[a..b]$ and $a \leq b$ ".

When the loop completes, $a \geq b$ (since the loop condition failed) and $a \leq b$ (by the loop invariant). Therefore $a = b$, and by the first part of the loop invariant the maximum element of $V[1..n]$ is equal to $V[a]$.

We use induction to prove the correctness of the invariant. Initially, $a = 1$ and $b = n$, so, the invariant trivially holds. Suppose that the invariant holds at the start of the loop. Then, we know that the maximum element of $V[1..n]$ is in $V[a..b]$. Notice that $V[a..b]$ is unimodal as well. If $V[mid] < V[mid + 1]$, then the maximum element of $V[a..b]$ occurs in $V[mid + 1..b]$ by case 1. Hence, after $a \leftarrow mid + 1$ and b remains unchanged in line 4, the maximum element is again in $V[a..b]$. The other case is symmetric.

To complete the proof, we need to show that the second part of the invariant $a \leq b$ is also true. At the start of the loop $a < b$. Therefore, $a \leq \lfloor (a + b)/2 \rfloor < b$. This means that $a \leq mid < b$ such that after line 4 or line 5 in which a and b get updated $a \leq b$ holds once more.

The divide and conquer approach leads to a running time of $T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$.

b- Give a divide and conquer algorithm to find the vertex with the maximum y coordinate in $O(\log n)$ time.

After finding the vertex $V[max]$ with the maximum x -coordinate, notice that the y -coordinates in $V[max], V[max + 1], \dots, V[n - 1], V[n], V[1]$ form a unimodal array and the maximum y -coordinate of $V[1..n]$ lies in this array. Thus the divide and conquer solution in part a can be used to find the vertex with the maximum y -coordinate. The total running time is $\Theta(\log n)$.

4) 15 pts

You are given a weighted directed graph $G = (V, E, w)$ and the shortest path distances $\delta(s, u)$ from a source vertex s to every other vertex in G . However, you are not given $\pi(u)$ (the predecessor pointers). With this information, give an algorithm to find a shortest path from s to a given vertex t in $O(|V| + |E|)$ time.

Start at u . Of the edges that point to u , at least one of them will come from a vertex v that satisfies $\delta(s, v) + w(v, u) = \delta(s, u)$. Such a v is on the shortest path. Recursively find the shortest path from s to v .

This algorithm hits every vertex and edge at most once, for a running time of $O(|V| + |E|)$.

5) 15 pts

Suppose you are choosing between the following three algorithms:

- Algorithm A solves problems of size n by dividing them into five subproblems of half the size, recursively solving each subproblem, and then combining the solutions in linear time.
- Algorithm B solves problems of size n by recursively solving two subproblems of size $n - 1$ and then combining the solutions in constant time.
- Algorithm C solves problems of size n by dividing them into nine subproblems of size $n/3$, recursively solving each subproblem, and then combining the solutions in $O(n^2)$ time.

What are the running times of each of these algorithms (in big-O notation), and which would you choose?

Algorithm A solves problems by dividing them into five subproblems of half the size, recursively solving each subproblem, and then combining the solutions in linear time.

$$T(n) = 5 T(n/2) + c n$$

Applying master theorem, $a=2$, $b=5$, $f(n)=c n$, $\text{degree}(f(n))=1$

$$\text{Since } \log_2 5 > 1, T(n) = O(n^{\log_a b}) = O(n^{\log_2 5})$$

Algorithm B solves problems of size n by recursively solving two subproblems of size $n-1$ and then combining the solutions in constant time.

$$T(n) = 2 T(n-1) + c = 2^2 T(n-2) + 2c + c = (2^n - 1) c$$

$$T(n) = O(2^n)$$

Algorithm C solves problems of size n by dividing them into nine subproblems of size $n/3$, recursively solving each subproblems and then combining the solution in $O(n^2)$ time.

$$T(n) = 9 T(n/3) + c n^2$$

Applying master theorem, $a=3$, $b=9$, $f(n)=c n^2$, $\text{degree}(f(n))=2$

$$\text{Since } \log_3 9 = 2, T(n) = O(n^2 \log n)$$

From above three algorithms, we can see that time complexity of the third algorithm is best. Thus, we will choose algorithm C.

6) 20 pts

- a- Suppose we are given an instance of the Shortest Path problem with source vertex s on a directed graph G . Assume that all edges costs are positive and distinct. Let P be a minimum cost path from s to t . Now suppose that we replace each edge cost c_e by its square root, $c_e^{1/2}$, thereby creating a new instance of the problem with the same graph but different costs.

Prove or disprove: P still a minimum-cost $s - t$ path for this new instance.

The statement can be disproved by giving a counterexample as follows.

$G=(V, E); V=\{s, a, t\}; E=\{(s, a), (a, t), (s, t)\};$

$\text{cost}((s, a))=9; \text{cost}((a, t))=16; \text{cost}((s, t))=36.$

It is obvious that the minimum-cost $s - t$ path is $s - a - t$.

By replacing each edge cost c_e by its square root, $c_e^{1/2}$, the costs become:

$\text{cost}((s, a))=3; \text{cost}((a, t))=4; \text{cost}((s, t))=6.$

Now the minimum-cost $s - t$ path is $s - t$, not $s - a - t$ anymore.

- b- Suppose we are given an instance of the Minimum Spanning Tree problem on an undirected graph G . Assume that all edges costs are positive and distinct. Let T be an MST in G . Now suppose that we replace each edge cost c_e by its square root, $c_e^{1/2}$, thereby creating a new instance of the problem (G') with the same graph but different costs.

Prove or disprove: T is still an MST in G' .

The statement is true due to that replacing each edge cost c_e by its square root, $c_e^{1/2}$ does not change the order of the cost, i.e. for positive real numbers a and b , if a is greater than b , $a^{1/2}$ is greater than $b^{1/2}$.

CS570
Analysis of Algorithms
Fall 2014
Exam I

Name: _____

Student ID: _____

Wednesday Evening Section

	Maximum	Received
Problem 1	20	
Problem 2	16	
Problem 3	16	
Problem 4	16	
Problem 5	16	
Problem 6	16	
Total	100	

2 hr exam

Close book and notes

If a description to an algorithm is required please limit your description to within 150 words, anything beyond 150 words will not be considered.

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[**TRUE/FALSE**]

Let T be a complete binary tree with n nodes. Finding a path from the root of T to a given vertex $v \in T$ using breadth-first search takes $O(\log n)$.

False :

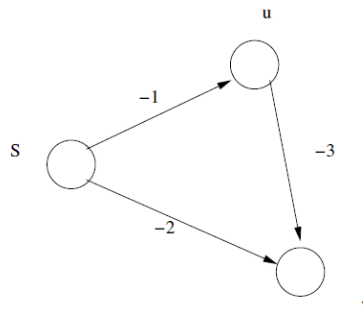
BFS requires $O(n)$ time. BFS examines each node in the tree in breadth-first order. The vertex v could well be the last vertex explored (Also, notice that T is not necessarily stored).

[**TRUE/FALSE**]

Dijkstra's algorithm works correctly on a directed acyclic graph even when there are negative-weight edges.

False:

Consider Dijkstra's algorithm run from source s in the following graph. The vertex v will be removed from the queue with $d[v] = -2$ even though the shortest path to it is -4 .



[**TRUE/FALSE**]

If the edge e is not part of any MST of G , then it must be the maximum weight edge on some cycle in G .

True:

Take any MST T . Since e is not part of it, e must complete some cycle T . e must be the heaviest edge on that cycle. Otherwise a smaller weight tree could be constructed by swapping the heavier edge on the cycle with e and thus T cannot be MST.

[**TRUE/FALSE**]

If $f(n) = O(g(n))$ and $g(n) = O(f(n))$ then $f(n) = g(n)$.

False:

$f(n) = n$ and $g(n) = n + 1$.

[TRUE/FALSE]

The following array is a max heap: [10; 3; 5; 1; 4; 2].

False: The element 3 is smaller than its child 4, violating the maxheap property.

[TRUE/FALSE]

There are at least 2 distinct solutions to the stable matching problem: one that is preferred by men and one that is preferred by women.

False:

Consider the example: two men X and Y; two women A and B.

Preference list (decreasing order):

X's list: A, B ; A's list: X, Y;

Y's list: A, B; B's list: X, Y.

The matching is unique: X-A and Y-B

[TRUE/FALSE]

In a binary max-heap with n elements, the time complexity of finding the second largest element is $O(1)$.

True:

The second largest one should be the larger (or equal) one of the two children of the root node. Finding them takes time $O(1)$. Note: it is possible that several elements have the same value including the first two layer of elements, then in this case, the largest element is equal to the second largest element, and returning one of the second layer elements is also fine.

[TRUE/FALSE]

Given a binary max-heap with n elements, the time complexity of finding the smallest element is $O(\lg n)$.

False:

The smallest element should be among the leaf nodes. Consider a full binary tree of n nodes. It has $(n+1)/2$ leafs (you can think of why). Then the worst case of finding the smallest element of a full binary tree (heap) is $\Theta(n)$

[TRUE/FALSE]

Kruskal's algorithm can fail in the presence of negative cost edges.

False:

The MST problem does not change even with the negative cost edges. So the Kruskal algorithm still works.

[TRUE/FALSE]

If a weighted undirected graph has two MSTs, then its vertex set can be partitioned into two, such that the minimum weight edge crossing the partition is not unique.

True: Let T_1 and T_2 be two distinct MSTs. Let e_1 be an edge that is in T_1 but not in T_2 . Removing e_1 from T_1 partitions the vertex set into two connected components.

There is a unique edge (call it e_2) that is in T_2 that crosses this partition. By minimum-weight property of T_1 and T_2 , both e_1 and e_2 should be of the same weight and further should be of the minimum weight among all edges crossing this partition.

2) 16 pts

The diameter of a graph is the maximum of the shortest paths' lengths between all pairs of nodes in graph G . Design an algorithm which computes the diameter of a connected, undirected, unweighted graph in $O(mn)$ time, and explain why it has that runtime.

Solution:

Suppose we suspected that a particular vertex v was an endpoint of the diameter of the graph. Since this graph is unweighted graph, we can use BFS to find the shortest path from any particular node v and report the largest layer number reached.

However, while we don't know any particular vertex v to be part of this, we do know that there is at least one vertex in the graph for which this is true. Accordingly, we can run BFS from every vertex v , and report the maximum layer reached in all of these runs.

The total time taken is $O(n(m + n))$; $O(m + n)$ for a single breadth-first search, which is run $O(n)$ times.

But notice that, when the graph is connected, n is $O(m)$. Accordingly, $O(m + n)$ is $O(m)$, for a total runtime of $O(mn)$, as desired.

3) 16 pts

Consider a stable marriage problem where the set of men is given by $M = \{m_1, m_2, \dots, m_N\}$ and the set of women is $W = \{w_1, w_2, \dots, w_N\}$. Consider their preference lists to have the following properties:

$$\forall w_i \in W: w_i \text{ prefers } m_i \text{ over } m_j, \forall j > i$$

$$\forall m_i \in M: m_i \text{ prefers } w_i \text{ over } w_j, \forall j > i$$

Prove that a unique stable matching exists for this problem.

Note: symbol $\forall$ means “for all”.

We will prove that matching S where m_i is matched to w_i for all $i = 1, 2, \dots, N$ is the unique stable matching in this case. We will use the notation $S(a) = b$ to denote that in matching S , a is matched to b .

It is evident that S is a matching because every man is paired with exactly one woman and vice versa. If any man m_j prefers w_k to w_j where $k < j$, then such a higher ranked woman w_k prefers her current partner to m_j . Thus, there are no instabilities and S is a stable matching. Now let's prove that this stable matching is unique.

By way of contradiction, let's assume that another stable matching S' , which is different from S , exists. Therefore, there must exist some i for which $S'(w_i) = m_k, k \neq i$. Let x be the minimum value of such an i . Similarly, there must exist some j for which $S'(m_j) = w_l, j \neq l$. Let y be the minimum value of such a j . Since $S'(w_i) = m_i$ for all $i < x$, and $S'(m_j) = w_j$ for all $j < y$, $x = y$. $S'(w_x) = m_k$ implies $x < k$. Similarly, $S'(m_y) = w_l$ implies that $y = x < l$. Given the preference lists, $m_y = m_x$ prefers w_x to w_l , and w_x prefers m_x to m_k . This is an instability and hence, S' cannot be a stable matching.

4) 16 pts

Ivan is a businessman and he has several large containers of fruits to ship. As he has only one ship he can transport only one container at a time and also it takes certain fixed amount of time per trip. As there are several varieties of fruits in the containers, the cost and depreciation factor associated with each container is different. He has n containers, $a_1, a_2, \dots, a_n$. Initial value of each container is $v_1, v_2, \dots, v_n$ and depreciation factor of each container is $d_1, d_2, \dots, d_n$. So, if container a_i happens to be the j^{th} shipment, its value will be $v_i / (d_i \times j)$. Can you help Ivan maximize total value of containers after depreciation ($\sum v_i / (d_i \times j)$) by providing an efficient algorithm to ship the containers? Provide proof of correctness and state the complexity of your algorithm.

Solution: Ship the containers in decreasing order of v_i / d_i . We prove the optimality of this algorithm by an exchange argument.

Thus, consider any other schedule. This schedule should consist an inversion. Further, in fact, there must be an adjacent such pair i, j . Note that for this pair we have

$v_i / d_i \leq v_j / d_j$, for $i < j$. If we can show that swapping this pair i, j does not decrease the total value, then we can iteratively do this until there are no more inversions, arriving at the greedy schedule without having decreased the value we are trying to maximize. It will then follow that our greedy algorithm is optimal.

Consider the effect of swapping i and j . The schedule of shipping all the other containers remains the same. Before the swap, the contribution of i and j to the total value was $(v_i / (d_i \times i) + v_j / (d_j \times j))$, while after the swap the total value is $(v_i / (d_i \times j) + v_j / (d_j \times i))$. The difference between value after the swap is $(v_j / d_j - v_i / d_i) \times (1/i - 1/j)$. As both, $(v_j / d_j - v_i / d_i)$ and $(1/i - 1/j)$ terms are positive, the total value is not decreased by swapping as desired.

The complexity of this solution is $O(n \log n)$, for sorting in decreasing order.

5) 16 pts

Consider an undirected graph $G = (V, E)$ with distinct nonnegative edge weights $w_e \geq 0$. Suppose that you have computed a minimum spanning tree of G . Now suppose each edge weight is increased by 1: the new weights are $w'_e = w_e + 1$. Does the minimum spanning tree change? Give an example where it changes or prove it cannot change.

Solution:

This question is similar to 6b question in your hw4. Monotonicity is preserved in $(+,.)$ function as in $(.)^2$

6) 16 pts

Mark all the correct statements and provide a brief explanation for each.

a. Consider these two statements about a connected undirected graph with V vertices and E edges:

I. $O(V) = O(E)$

II. $O(E) = O(V^2)$

(a) I and II are both false.

(b) Only I is true.

(c) Only II is true.

(d) I and II are both true.

(d) If there are V vertices, there can be at most V^2 edges in the graph. For the graph to be connected, there must be at least $V - 1$ edges.

$$E \leq V(V-1)/2 = O(V^2).$$

$$\text{Also, } V \leq 2(V-1) \leq 2E$$

$$\Rightarrow V = O(E)$$

b. Suppose the shortest path from node i to node j goes through node k and that the cost of the subpath from i to k is D_{ik} . Consider these two statements:

I. Every shortest path from i to j must go through k .

II. Every shortest path from i to k has cost D_{ik} .

(a) I and II are both true.

(b) Only I is true.

(c) Only II is true.

(d) I and II are both false.

(c) I. is false because there can be multiple shortest paths from i to j , not necessarily going through k . For example, there could be an edge between i and j that has the same cost as the path through k .

II. is true because if there were a path P' from i to k that had cost less than D_{ik} , then the path from i to j consisting of P' and the path from k to j would be shorter than the shortest i - j path, which is a contradiction.

c. Consider the execution times of two algorithms I and II:

I. $O(n \log n)$

II. $O(\log(n^n))$

- (a) Only I is polynomial.
- (b) Only II is polynomial.
- (c) I and II are both polynomial.
- (d) Neither I nor II is polynomial.

(c) $\log(n^n) = n \log n \leq n^2$

d. Suppose $f(n) = 3n^3 + 2n^2 + n$. Consider these statements:

I. $f(n) = O(n^3)$

II. $f(n) = O(n^4)$

- (a) Only I is true.
- (b) Only II is true.
- (c) I and II are both true.
- (d) I and II are both false.

(c) For all $n \geq 1$,
 $f(n) \leq 3n^3 + 2n^3 + n^3 = 6n^3 \leq 6n^4$

Additional Space

CS570
Analysis of Algorithms
Fall 2014
Exam I

Name: _____

Student ID: _____

Thursday Evening Section

DEN Yes / No

	Maximum	Received
Problem 1	20	
Problem 2	15	
Problem 3	15	
Problem 4	20	
Problem 5	15	
Problem 6	15	
Total	100	

2 hr exam

Close book and notes

If a description to an algorithm is required please limit your description to within 150 words, anything beyond 150 words will not be considered.

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[**TRUE/FALSE**]

Adding a number w on the weight of every edge of a graph might change the shortest path between two vertices u and v .

True:

Consider $G = (V, E)$ with $V = \{u, x, v\}$ and $E = \{ux, xv, uv\}$. Let weight of edge $ux = w_{ux} = w_{xv} = 3$ and $w_{uv} = 7$. Then the shortest path from u to v is given by $u \rightarrow x \rightarrow v$ with a total weight of 6. However, if we now add 2 to every edge weight, the path $u \rightarrow x \rightarrow v$ will have a total weight of 10 while the path $u \rightarrow v$ will have a weight of 9, making it the new shortest path.

[**TRUE/FALSE**]

Suppose that for some graph G we have that the average edge weight is A . Then a minimum spanning tree of G will have weight at most $(n - 1) \cdot A$.

False:

We may be forced to select edges with weight much higher than average. For example, consider a graph G consisting of a complete graph G' on 4 nodes, with all edges having weight 1 and another vertex u , connected to one of the vertices of G' by an edge of weight 8. The average weight is $(8 + 6)/7 = 2$. Therefore, we would expect the spanning tree to have weight at most $4 \cdot 2 = 8$. But the spanning tree has weight more than 8 because the unique edge incident on u must be selected.

[**TRUE/FALSE**]

DFS finds the longest paths from start vertex s to each vertex v in the graph.

False:

Depends on the order in which the nodes are traversed

[**TRUE/FALSE**]

If one can reach every vertex from a start vertex s in a directed graph, then the graph is strongly connected.

False

[**TRUE/FALSE**]

$f(n) = 4n + 3\sqrt{n}$ is both $O(n)$ and $\Omega(n)$.

True:

The dominant term is $4n$, which is obviously both $O(n)$ and $\Omega(n)$

[TRUE/FALSE]

In Fibonacci heaps, the decrease-key operation takes $O(1)$ time.

True:

Just as definition of decrease-key operation in Fibonacci heaps

[TRUE/FALSE]

If the edge weights of a weighted graph are doubled, then the number of minimum spanning trees of the graph remains unchanged.

True:

With edge weights doubled, the weights of all possible spanning trees in the graph are doubled. So any MST in the original graph is also the MST in the new graph; any spanning tree that is not the MST in the original graph is still not the MST in the new graph.

[TRUE/FALSE]

Given a binary max-heap with n elements, the time complexity of finding the smallest element is $O(\lg n)$.

False:

The smallest element should be among the leaf nodes. Consider a full binary tree of n nodes. It has $(n+1)/2$ leafs (you can think of why). Then the worst case of finding the smallest element of a full binary tree (heap) is $\Theta(n)$

[TRUE/FALSE]

An undirected graph $G = (V, E)$ must be connected if $|E| > |V| - 1$

False:

Consider a graph having nodes: a, b, c, d, e . $\{a, b, c, d\}$ forms a fully connected subgraph, while e is isolated from other nodes. Now the fully connected subgraph has 6 edges, and there are only 5 nodes in total. But this graph is not a connected graph.

[TRUE/FALSE]

If all edges in a connected undirected graph have unit cost, then you can find the MST using BFS.

True:

Any spanning tree of a graph having only unit cost edges is also a MST, because the weight is always $n-1$ units. Of course, BFS gives a spanning tree in the connected graph.

2) 16 pts

At the Perfect Programming Company, programmers program in pairs in order to ensure that the highest quality code is produced. The productivity of each pair of programmers is the speed of the slower programmer. For an even number of programmers, give an efficient algorithm for pairing them up so that the sum of the productivity of all pairs is maximized. Analyze the running time and prove the correctness of your algorithm.

Solution:

A simple greedy algorithm works for this problem. Sort the speeds of the programmers in decreasing order using an optimal sorting algorithm such as merge sort. Consecutive sorted programmers are then paired together starting with pairing the fastest programmer with the second fastest programmer.

Sorting takes $O(n \lg n)$ time while pairing the programmers takes $O(n)$ time giving a total running time of $O(n \lg n)$.

Correctness: Let P be the set of programmers. The problem exhibits an optimal substructure.

Assume the optimal pairing. Given any pair of programmers $(i; j)$ in the optimal pairing, the optimal sum of productivity is just the sum of the productivity of $(i; j)$ with the optimal sum of the productivity of the all pairs in $P - \{i, j\}$.

We now show that the greedy choice works by showing that there exists an optimal pairing such that the two fastest programmers are paired together. Assume an optimal pairing where fastest programmer i is not paired with the second fastest programmer j . Instead let i be paired with k and j be paired with l . Let p_i, p_j, p_k and p_l be the programming speeds of i, j, k and l respectively. We now change the pairings by pairing i with j and k with l . The change in the sum of productivities is

$$(p_j + \min(p_k, p_l)) - (p_k + p_l) = p_j - \max(p_k, p_l) \geq 0$$

since p_j is at least as large as the larger of p_k and p_l . We now have an optimal pairing where the fastest programmer is paired with the second fastest programmer. Hence to find the optimal solution, we can keep pairing the two fastest remaining programmers together.

3) 16 pts

The graph K_n is defined to be an undirected graph with n vertices and all possible edges (a fully connected graph). That is, the vertices are named $\{1, \dots, n\}$ and for any numbers i and j with $i \neq j$, there is an edge between vertex i and vertex j . Describe the result of a breadth-first search and a depth-first search of K_n . For each search, describe the resulting search tree.

Solution:

For the BFS, the algorithm looks at all the neighbors of the root node before going to the second level. Since every node in the graph is a neighbor of every other node, every node other than the root is put at level 1. Thus the tree has one node at level 0 and $n-1$ nodes at level 1. The non-tree edges are exactly those edges between different nodes on level 1 -- there are $(n-1 \text{ choose } 2)$ of these.

For the DFS, the search of the root node (call it 1) will find another node 2, and then the recursive call on node 2 will find node 3, and so forth. None of the recursive calls can terminate while any undiscovered nodes remain. So the tree edges form a single path of $n-1$ edges, with one node on each level and thus a single leaf. There are back edges, $(n-1 \text{ choose } 2)$ of them, as each node has a back edge to each of its ancestors except its parent.

4) 16 pts

A d -ary heap is like a binary heap, but instead of 2 children, nodes have d children.

(a) How would you represent a d -ary heap in an array? [4 points]

If we know the maximum number, N , of nodes that we may store in the d -ary heap, we can allocate an array H of size N indexed by $i = 1, 2, \dots, N$. The heap nodes will correspond to positions in H . $H[1]$ will be the root node and for any node at position i in H , its children will be at positions $d*(i-1) + 2, d*(i-1) + 3, \dots, d*(i-1) + d + 1$, and the parent position at $\lfloor (i-2)/d \rfloor + 1$. If there are $n < N$ elements in the heap at any time, we will use the first n indices of the array to store the heap elements.

(b) What is the height of a d -ary heap of n elements in terms of n and d ? [4 points]

There is 1 node at level 0 (L_0), d at L_1 , d^2 at L_2 and so on. Let h be the height of the d -ary heap. Then there are d^h nodes at the last level of the heap. Thus,

$$n = 1 + d + d^2 + d^3 + \dots + d^h$$

Solving this, we get

$$h = \log_d(n(d-1)+1) - 1$$

(c) Give an efficient implementation of ExtractMin. Analyze its running time in terms of d and n . [8 points]

Similar to the ExtractMin operation for a binary heap, for a d -ary heap with n elements, we find the minimum (the root node) in $O(1)$ time and to delete it, we replace $H[1]$ with $H[n] = w$. If the resulting array is not a heap, we use Heapify-up or Heapify-down to fix the heap in $O(h) = O(\log_d n)$ time.

5) 16 pts

You are given a directed graph representing several career paths available in the industry. Each node represents a position and there is an edge from node v to node u if and only if v is a pre-requisite for u . Starting positions are the ones which have no pre-requisite positions. Except starting positions, we can only perform a position only if any of its pre-requisite positions are performed. Top positions are the ones which are not pre-requisites for any positions. Ivan wants to start a career at any of the starting positions and achieve a top position by going through the minimum number of positions. Using the given graph provide a linear time algorithm to help Ivan choose his desired career path. Note that this graph has no cycles.

Solution: Add a node s and connect an edge from s to all nodes that have no incoming edges (starting positions). Add a node t and connect an edge from all nodes with no outgoing edges (top positions) to t . Do a BFS from s and find the shortest path to t . This will give you the shortest path from a starting position to a top position in $O(m+n)$.

6) 16 pts

Suppose we are given an instance of the Minimum Spanning Tree problem on a graph G .

Assume that all edges costs are distinct. Let T be a minimum spanning tree for this instance. Now suppose that we replace each edge cost c_e by its square, c_e^2 thereby creating a new instance of the problem with the same graph but different costs.

Prove or disprove: T is still a MST for this new instance.

Solution:

The claim is false.

Note that the edge cost can be negative. (This is the key difference between this problem and a homework problem in HW4)

Consider the following example:

The original graph: $G = (V, E)$ being an undirected graph, where $V = \{a, b, c\}$, $E = \{(a,b), (b,c), (a,c)\}$. Weights: $w(a,b) = 1$, $w(b,c) = -3$, $w(a,c) = 2$.

The MST is : $T = (T_v, T_E)$, where $T_v = \{a, b, c\}$, $T_E = \{(a,b), (b,c)\}$

After squaring the edge weights, we get a new graph G' with $w'(a,b) = 1$, $w'(b,c) = 9$, $w'(a,c) = 4$.

The MST is: $T' = (T'_v, T'_E)$, where $T'_v = \{a, b, c\}$, $T'_E = \{(a,b), (a,c)\}$.

So the MST changes..

Additional Space

CS570
Analysis of Algorithms
Spring 2007
Exam 1

Name: _____

Student ID: _____

	Maximum	Received
Problem 1	14	
Problem 2	6	
Problem 3	15	
Problem 4	20	
Problem 5	15	
Problem 6	20	
Problem 7	10	

Note: The exam is closed book closed notes.

1) 14 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[**TRUE/FALSE**] **False**

Function $f(n) = 5n^3 2^n + 6n^2 3^n$ is $O(n^3 2^n)$.

[**TRUE/FALSE**] **True**

$n \log^3 n$ is NOT $O(n \log n)$

[**TRUE/FALSE**] **True**

In an undirected graph, the shortest path between two nodes lies on some minimum spanning tree.

[**TRUE/FALSE**] **False**

Max base heap can be converted into Min based heap by reversing the order of the elements in the heap array.

[**TRUE/FALSE**] **False**

Kruskal's algorithm for finding a MST of a weighted, undirected graph is a form of a dynamic programming technique.

[**TRUE/FALSE**] **True**

In the case of applying Dijkstra's algorithm for dense graph, Fibonacci implementation is the best one among Binary, Binomial, and Fibonacci.

[**TRUE/FALSE**] **False**

If all of the weights from a connected and weighted graph are distinct, then distinct spanning trees of the graph have distinct weights.

2) 6pts

What is the worst-case complexity of the each of the following code fragments?

a)

```
for (i = 0; i < 2N; i++) {  
    sequence of statements  
}  
for (j = 0; j < 3M; j++) {  
    sequence of statements  
}  
O(M+N)
```

b)

```
for (i = 0; i < N; i++) {  
    for (j = 0; j < 2N2; j++) {  
        sequence of statements  
    }  
}  
for (k = 0; k < 3M; k++) {  
    sequence of statements  
}  
O(N3+M)
```

c)

```
for (i = 0; i < N3; i++) {  
    for (j = i; j < 2lg(M); j++) {  
        sequence of statements  
    }  
}  
for (k = 0; k < M; k++) {  
    sequence of statements  
}
```

$O(N^3 \log M + M)$

3) 15 pts

The maximum spanning tree problem is to find a spanning tree of a graph whose edge weights have the largest sum possible. Give an algorithm to find a maximum spanning tree and give a proof that the algorithm is correct.

Solution: We can reduce finding the maximum spanning tree problem into finding minimum spanning tree problem. The reduction is as follows: Let $w(e)$ denote the weight of edge e in the graph and let M be the maximum weight of all the edges in G . Now consider re-weighting each edge e in G as $M - w(e)$. Now consider two spanning tree of G , T and T' and note that they both contain exactly the same number of edges, i.e., $n-1$ edges, where G has n nodes. The key point is that $\sum_{e \in T} w(e) \geq \sum_{e' \in T'} w(e')$ if and only if

$$\sum_{e \in T} [M - w(e)] = (n-1)M - \sum_{e \in T} w(e) \leq (n-1)M - \sum_{e' \in T'} w(e') = \sum_{e' \in T'} [M - w(e')]$$

Hence, if we solve the minimum spanning tree problem using the new edge weights, the optimal tree found will be the maximum spanning tree using the original weights. Therefore we can reduce the maximum spanning tree problem into the minimum spanning tree problem. Hence we can use either Kruskal's algorithm or Prim's algorithm after the reduction and the running time is

$O(m \log n)$

4) 20 pts

Suppose you are given two set A and B , each containing n positive integers. You can choose to reorder each set however you like. After reordering, let a_i be the i -th element of set A , and let b_i be the i th element of set B . You then receive a payoff of $\sum_{i=1}^n b_i \log a_i$. Give an algorithm that will maximize your payoff. Prove that your algorithm maximizes the payoff, and state its running time.

Algorithm: Sort A and B in increasing order and the payoff is maximized

Proof: Suppose $\{a_1, \dots, a_n\}$ and $\{b_1, \dots, b_n\}$ is an optimal solution but $(a_i > a_j)$ and $(b_i < b_j)$. Then switch a_i and a_j we can get a better solution. The reason is as follows:

$$\begin{aligned} a_i > a_j, b_i < b_j &\Leftrightarrow a_i^{b_j - b_i} > a_j^{b_j - b_i} \Leftrightarrow a_i^{b_j} a_j^{b_i} > a_j^{b_j} a_i^{b_i} \Leftrightarrow \log(a_i^{b_j} a_j^{b_i}) > \log(a_j^{b_j} a_i^{b_i}) \\ &\Leftrightarrow b_j \log a_i + b_i \log a_j > b_j \log a_j + b_i \log a_i \end{aligned}$$

which contradicts that $\{a_1, \dots, a_n\}$ and $\{b_1, \dots, b_n\}$ is an optimal solution

Running time: Sorting takes $O(n \log n)$. Hence the running time of our algorithm is $O(n \log n)$

5) 15 pts

Given a connected graph $G = (V, E)$ with positive edge weights and two nodes s, t in V , prove or disprove:

- a. If all edge weights are unique, then there is a single shortest path between any two nodes in V .
 - b. If each edge's weight is increased by k , the shortest path cost will increase by a multiple of k .
 - c. If the weight of some edge e decreases by k , then the shortest path cost will decrease by at most k .
-
- a) False. Counter Example: (s, a) with weight 1, (a, t) with weight 2 and (s, t) with weight 3. There are two shortest path from s to t though the edge weights are unique.
 - b) False. Example: suppose the shortest path $s \rightarrow t$ consist of two edges, each with cost 1, and there is also an edge $e=(s, t)$ in G with $\text{cost}(e)=3$. If now we increase the cost of each edge by 2, e will become the shortest path (with the total cost of 5).**
 - c) True. For any two nodes s, t , assume that $P_1, \dots, P_k$ are all the paths from s to t . If e belongs to P_i then the path cost decrease by k , otherwise the path cost unchanged. Hence all paths from s to t will decrease by at most k . As shortest path is among them, then the shortest path cost will decrease by at most k .

Grading policy: 5 points for each item. 2 points for TRUE/FALSE part and 3 points for prove/disprove part.

6) 20 pts

Sam is shocked to find out that his word processor has corrupted his text document and has eliminated all spacing between words and all punctuation so his final term paper looks something like: “anawardwinningalgorithmto...”. Luckily he has access to a Boolean function *dictionary*() that takes a string *s* and returns true if *s* is a valid word and false otherwise. Considering that he has limited time to turn in his paper before the due date, find an algorithm that reconstructs his paper into a sequence of valid words with no more than $O(n^2)$ calls to the *dictionary*() function.

Solution: We denote the string as $s[1], \dots, s[n]$. For the sub string $s[1], \dots, s[m]$, we construct the table *P* as follows: if $s[1], \dots, s[m]$ is composed of a sequence of valid words, then $P[m]=\text{true}$; otherwise $P[m]=\text{false}$. The recursive relation is as follows:

$$P[0] = \text{true}$$

$$P[m] = \bigvee_{0 \leq i \leq m-1} (P[i] \wedge \text{dictionary}(s[i+1], \dots, s[m]))$$

If $P[m]$ is true, we denote $q[m]=j$ where j is such that $P[j] \wedge \text{dictionary}(s[j+1], \dots, s[m])$ is true. By the definition of $P[m]$, it is obvious that such a j must exist when $P[m]$ is true. To reconstruct the paper into a sequence of valid words, we just need to output $n, q[n], q[q[n]], \dots, 0$ as segmentation points.

Running time: The length of table of *P* and *q* is n and each step when we compute $P[m]$ we need at most m calls of function *dictionary*(). Note the $m \leq n$. Hence the running time is bounded by $O(n^2)$

Remark: Actually this question is exactly the same as the first problem in HW4: Problem 5 in Chapter 6 if we set $quality(Y_{i+1}, k) = 0$ if $dictionary(Y_{i+1}k) = \text{false}$ and $quality(Y_{i+1}, k) = 1$ if $dictionary(Y_{i+1}k) = \text{true}$.

7) 10 pts

An $n \times n$ array A consists of 1's and 0's such that, in any row of A , all the 1's come before any 0's in that row. Give the most efficient algorithm you can for counting the number of 1's in A .

Algorithm: For each row i , we use binary search to find the index of last element of 1 in the array $A_i[1, \dots, n]$. Suppose the index is a_i . Then the sum of $a_i (0 \leq i < n+1)$ is the number of 1's in A .

Proof: Assume that in i -th row k is the index of last element of 1. Then we have the property that $a_{ij}=1$ for $j \leq k$ and $a_{ij}=0$ for $j > k$ as all 1's come before any 0's in each row. During the binary search, if $a_{im}=1$, then we must have $k \geq m$; $a_{im}=0$, then we must have $k < m$. By doing this, we can finally find the value of k for each row.

Running time: There are n rows and for each row the running time to find the last element which is 1 by binary search is $\log n$. Hence the running time of our algorithm is $O(n \log n)$

Grading policy: 5 points for algorithm. 3 points for proof and 2 points for running time.

Additional Space

Additional Space

CS570

Analysis of Algorithms

Spring 2009

Exam I- Solution

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[**TRUE/FALSE**]

Given a weighted graph and two nodes, it is possible to list all shortest paths between these two nodes in polynomial time.

False. Not valid for graph with negative weights.

[**TRUE/FALSE**]

Given a graph $G = (V, E)$ with cost on edges and a set S which is a subset of V , let (u, v) be an edge such that (u, v) is the minimum cost edge between any vertex in S and any vertex in $V - S$. Then, the minimum spanning tree of G must include the edge (u, v) .

True. Refer to textbook, page 145, theorem 4.17.

[**TRUE/FALSE**]

Let $G = (V, E)$ be a weighted graph and let M be a minimum spanning tree of G . The path in M between any pair of vertices v_1 and v_2 must be a shortest path in G .

False. Counter example: a loop with three nodes A, B and C . The edge costs are $A-B: 1, B-C: 1, A-C: 1.5$. Edge $A-C$ is not included in M , but the shortest path between A and C is through edge $A-C$.

[**TRUE/FALSE**]

$2n^2 + \theta(n) = \theta(n^2)$

True.

[**TRUE/FALSE**]

Kruskal's algorithm can fail in the presence of negative cost edges.

This is False.

[**TRUE/FALSE**]

For any cycle in a graph, the cheapest edge in the cycle is in a minimum spanning tree.

False. If every edge in a cycle C is very expensive then it is possible that the MST doesn't use any edge in C (including the cheapest edge in C).

[TRUE/FALSE]

In every instance of the stable marriage problem there is a stable matching containing a pair (m, w) such that m is ranked first on the preference list of w and w is ranked first on the preference list of m .

False. Consider the following scenario where there are two men and women each in M and W with the following preferences:

m prefers w to w'

m' prefers w' to w

w prefers m' to m

w' prefers m to m'

[TRUE/FALSE]

Let $T(n)$ be a function obeying the recurrence $T(n) = 5T(n/5) + a$ with initial condition $T(1) = b$, where a and b are positive numbers. Then $T(n) = \Theta(n \log n)$.

False. By the Master Theorem $T(n) = \Theta(n)$. We compare $n \log_5^5$ with the overhead term of $a = O(n^0)$ and find that we are in the case where the former dominates.

[TRUE/FALSE]

If the edges in a connected undirected graph are unit cost, then you can find the MST using BFS.

This is true.

[TRUE/FALSE]

Asymptotically, a running time of n^d is better than $10n^d$

This is False. Asymptotically, both of them are the same Basically within theta of each other

2) 10 pts

Give an algorithm that given as input a sequence of n numbers and a natural number $k < n$, outputs the smallest k numbers of the sequence in $O(n + k \log n)$ time.

Solution: Building a MIN-HEAP takes $O(n)$ time and doing EXTRACT-MIN k times takes $O(k \log n)$ time and gives us the k smallest numbers of the sequence. Hence running time is $O(n + k \log n)$.

3) 10 pts

Arrange the following in the increasing order of asymptotic growth (x is a constant which is greater than 1).

$$x^{\sqrt{\log n}} \quad x^n \quad n^{10} \quad n \log n \log^4 n \quad n^{\log n} \quad x^{x^n} \quad x^{n^x}$$

Solution:

$$x^{\sqrt{\log n}} \quad n \log n \log^4 n \quad n^{10} \quad n^{\log n} \quad x^n \quad x^{n^x} \quad x^{x^n}$$

4) 20 pts

You have a sufficient supply of coins, and your goal is to be able to pay any amount using as few coins as possible.

(a) Describe a greedy algorithm to solve the problem when the coins used are of values 1, 5, 10 and 25 cents. Prove the optimality of the algorithm.

Solution: Let k be the number of coins of denomination d_i just enough to be greater than equal to d_{i+1} , where d_{i+1} is the next higher denomination. Then k coins of denomination d_i can be replaced by d_{i+1} and some other coin(s) using less than k coins. One can verify this observation by taking into account the fact that 5 pennies can be replaced by a nickel, 2 nickels can be replaced by a dime and 3 dimes can be replaced by a quarter and a nickel (2 coins).

Let the money to change for be n , $n = 25a + 10b + 5c + d$. a, b, c, d are the numbers of quarters, dimes, nickels and pennies. We have $b < 3$, $c < 2$ and $d < 5$ (or it will violate the conclusion in the observation, e.g., if $b \geq 3$, we can replace 3 dimes by a quarter and a nickel which results in less coins). We prove that when $n \geq 25$, we should always pick a quarter. As $10b + 5c + d \leq 10 \cdot 2 + 5 \cdot 1 + 1 \cdot 4 = 29$, it means in the optimal solution, the amount of the money of the dimes, nickels and pennies will be no more than 29 cents. So we only need to prove the cases when n is 25, 26, 27, 28 or 29. One can easily verify that in such cases it is always better to choose a quarter. Similarly we can prove that when $10 \leq n < 25$, we should always pick a dime and when $5 \leq n < 10$ we should always pick a nickel.

(b) Show that a greedy algorithm does not yield optimal results if you are using only coins of value 1 cent, 7 cents and 10 cents.

Solution: Counter example: Consider making change for $x = 14$. The greedy algorithm yields $1 \times (10 \text{ cents}) + 4 \times (1 \text{ cents})$. This is 5 coins. However, we can see that $2 \times (7 \text{ cents}) = 14 = x$ also. Here we use 2 coins. Obviously $2 < 5$, so Greedy is sub-optimal in the case of coin denominations 1, 7, and 10.

5) 10 pts

Let T be a minimum spanning tree for G with edge weights given by weight function w . Choose one edge $(x, y) \in T$ and a positive number k , and define the weight function w' by

$$w'(u, v) = w(u, v), \text{ if } (u, v) \neq (x, y)$$

$$w'(u, v) = w(u, v) - k, \text{ if } (u, v) = (x, y)$$

Show that T is also a minimum spanning tree for G with edge weights given by w' .

Solution: Proof by contradiction. Note T is a spanning tree for G with weight function w' and $w'(T) = w(T) - k$.

Suppose T is not a MST for G with w' and hence there exists a tree T' which is an MST for G with w' . So $w'(T') < w'(T)$ (*).

We have two cases to analyze. First, if $(x, y) \in T'$ then $w'(T') = w(T') - k$ and from the above we have $w(T') < w(T)$. Now T' is a spanning tree for G with w which is better than T which is a MST for G with w . A contradiction. Case 2, suppose $(x, y) \notin T'$ then $w'(T') = w(T')$ and so by (*) we have $w(T') < w'(T) = w(T) - k$. A similar contradiction again.

Hence T is a MST for G with w' .

6) 15 pts

Stable Matching: Prove that, if all boys have the same list of preferences, and all the girls have the same list preferences, there can be only one stable marriage.

Solution: Proof by contradiction. Suppose there are n boys $b_1, b_2, \dots, b_n$; and n girls $g_1, g_2, \dots, g_n$. Without loss of generality, assume that all the boys prefer g_1 over g_2 , g_2 over g_3 , and so on. Likewise all the girls most prefer b_1 and least prefer b_n . It is easy to verify that the pairing $(b_1, g_1), (b_2, g_2), \dots, (b_n, g_n)$ is a stable matching. Suppose that there is another stable matching, including couple (b_i, g_j) for $i \neq j$. If there are multiple such pairs, let i be the smallest such value. Therefore $j > i$, because g_1 to g_{i-1} are engaged to b_1 through b_{i-1} respectively: so j cannot be less than i . By the definition of the problem, everyone is engaged, including g_i , and thus for some $k > i$, our alternate pairing includes the couple (b_k, g_i) . Note that b_i is a more popular boy than b_k , since we chose i to be the smallest possible value. Therefore g_i prefers b_i over her fiancé. And because $i < j$, we know b_i prefers g_i over his fiancée. Thus b_i and g_i constitute an unstable couple. But this contradicts our supposition that this alternate matching is stable. So we conclude that there is no alternate stable matching.

7) 15 pts

Give a divide-and-conquer algorithm to find the average of n real numbers. You should clearly show all steps (divide, conquer, and combine) in your pseudo code. Analyze complexity of your solution.

Solution:

Divide: divide array up in 2 equal pieces

Conquer: solve the two subproblems recursively and return the average

Combine: Compute the average for the original problem by:

Average = (number of items in sub problem 1 * average for subproblem 1 + number of items in sub problem 2 * average for subproblem 2) / (size of the original problem)

Complexity of Divide is $O(1)$, complexity of Combine is $O(1)$. Master theorem gives you a complexity of $O(n)$ for the solution.

Question 1:

1. False. A bipartite graph cannot contain cycle of odd length, however the number of cycles could be odd. For instance, consider the graph with the edge set $\{(a,b),(b,c),(c,d),(d,a)\}$. The number of edges is 4, which is even.
2. True. A tree by definition does not contain cycles, and hence does not contain cycles of odd length and is bipartite.
3. True. Let T_1 and T_2 be two distinct MSTs. Hence there exists an edge e that is in T_1 but not T_2 . Add e to T_2 thereby inducing a (unique) cycle C . There are at least two edges of maximum weight in C (for if there were a unique maximum weight edge in C , then it cannot be in an MST thereby contradicting the fact that it is either in T_1 or T_2).
4. True. The claim in question follows from the following two facts: (i) the weight of a tree is linear in its edge weights and (ii) every spanning tree has exactly $|V|-1$ edges. Hence the weight of a spanning tree under the modification goes from $w(T)$ to $2w(T)$. Thus an MST of the original graph remains an MST even after the modification.
5. False. The max element in a min heap with n elements is one of the leafs. Since there are $\Theta(n)$ leafs and the structure of min heap does not carry any information on how the leaf values compare, to find a max element takes at least $\Omega(n)$ time.
6. False. Consider for instance $f(n) = n$ and $g(n)=n^2$. Clearly $n+n^2$ is not in $\Theta(n)$.
7. True.
8. True. The number of spanning trees of a completely connected graph with n vertices is n^{n-2} (Cayley's formula) which grows at least exponentially in n . Even without the knowledge of Cayley's formula, it is easy to derive $\Omega(2^n)$ lower bound.
9. True. If the edge lengths are identical, then BFS does find shortest paths from the source. BFS takes $O(|V|+|E|)$ where as Dijkstra even with a Fibonacci heap implementation takes $O(|E| + |V| \log |V|)$
10. False. The $O()$ notation merely gives an upper bound on the running time. To claim one is faster than other, you need an upper bound for the former and a lower bound (Ω) for the latter.

Question 2:

Algorithm1:

1. Construct the adjacent list of G^{rev} in order to facilitate the access to the incoming edges to each node.

2. Starting from node t , go through the incoming edges to t one by one to check if the following condition is satisfied:

$$\delta(s, t) == e(u, t) + \delta(s, u), \quad (u, t) \in E$$

3. There must be at least one node u satisfying the above condition, and this u must be a predecessor of node t .
4. Keep doing the same checking operation recursively on the predecessor node to get the further predecessor node until node s is reached.

Algorithm2:

Run a modified version of Dijkstra algorithm without using the priority queue. Specifically,

1. Set $S = \{s\}$.
 2. While (S does not include t)
 - Check each node u satisfying $u \notin S, (v, u) \in E, v \in S$, if the following condition is satisfied:

$$\delta(s, u) == e(v, u) + \delta(s, v)$$
 - If yes, add u into S , and v is the predecessor of u .
- endWhile

Complexity Analysis:

In either of the above algorithms, each directed edge is checked at most once, the complexity is $O(|V| + |E|)$.

Question 3:

Algorithm:

Sort jobs by starting time so that $s_1 \leq s_2 \leq \dots \leq s_n$.

Define $f_1, f_2, \dots, f_n$ as the corresponding finishing times of the jobs.

Define d as the number of allocated processor.

Initialize $d = 0$;

For $j = 1$ to n

 If (job j is compatible with processor)

 Allocate processor k to job j .

 Else

 Allocate a new processor $d + 1$;

 Schedule job j to processor $d + 1$;

Update $d = d + 1$;
EndFor

Implementation:

Sort starting time takes time $O(n \log n)$.

Keep the allocated processors in a priority queue (Min-heap), accessed by minimum finishing time.

For each processor k , maintain the finishing time of the last job added.

To check the compatibility: compare the job j 's starting time with the finishing time (key value of the root of the Min-heap) of processor k with last allocated job i . If $s_i > f_i$, allocate processor k to job j ; otherwise, allocate a new processor. The operation corresponds to either changing key value or adding a new node. Either operation takes time $O(\log n)$.

For n jobs, the total time is $O(n \log n)$.

Proof:

Definition: define the "depth" d of a set of jobs to be the maximum number that pass over any single point on the time-line.

Then we have the follow observations: All schedules use $\geq d$ (the depth) processors

- Assume that greedy uses D processors where $D > d$.
- Processor D is used because we needed to schedule a job, say j , that is incompatible with all $D - 1$ other processors.
- Since we sorted by starting time, all these incompatibilities are caused by jobs with start times $\geq s_j$
- The number of such incompatible processors is at most $d - 1$.
- Therefore, $D - 1 \leq d - 1$, a contradiction

Question 4:

- i) The mayor is merely contending that the graph of the city is a strongly-connected graph, denoted as G . Form a graph of the city (intersections become nodes, one-way streets become directed edges). Suppose there are n nodes and m edges. The algorithm is:
- (1) Choose an arbitrary node s in G , and run BFS from s . If all the nodes are reached the BFS tree rooted at s , then go to step (2), otherwise, the mayor's statement must be wrong. This operation takes time $O(m + n)$

- (2) Reverse the direction of all the edges to get the graph G^{inv} , this step takes time $O(m + n)$
- (3) Do BFS in G^{inv} starting from s . If all the nodes are reached, then the mayor's statement is correct; otherwise, the mayor's statement is wrong. This step takes time $O(m + n)$.

Explanation: BFS on G tests if s can reach all other nodes; BFS on G^{inv} tests if all other nodes reach s . If these two conditions are not satisfied, the mayor's statement is wrong obviously; if these two conditions are satisfied, any node v can reach any node u by going through s .

- ii) Now the mayor is contending that the intersections which are reachable from the city form a strongly-connected component. Run the first step of the previous algorithm (test to see which nodes are reachable from town hall). Remove any nodes which are unreachable from the town hall, and this is the component which the mayor is claiming is strongly connected. Run the previous algorithm on this component to verify it is strongly connected.

Question 5:

No, the modification does not work. Consider the following directed graph with edge set $\{(a,b),(b,c),(a,c)\}$ all of whose edge weights are -1 . The shortest path from a to c is $((a,b),(b,c))$ with path cost -2 . In this case, $w=1$ and if 2 is added to every edge length before running Dijkstra starting from a , then Dijkstra outputs (a,c) as the shortest path from a to c which is incorrect.

Another way to reason why the modification does not work is that if there were a directed cycle in the graph (reachable from the chosen source) whose total weight is negative, then the shortest paths are not well defined since you could traverse the negative cycle multiple times and make the length arbitrarily small. Under the modification, all edge lengths are positive and hence clearly the shortest paths in the original graph are not preserved.

Remark: Many students who claimed that the modification works made this common mistake. They claimed that since the transformation preserved the relative ordering of the edges by their weights, the algorithm should work. This is incorrect. Even though the edge lengths are increased by the same amount, the path lengths change as a function of the number of edges in them.

Question 6:

- a) We use the $\left\lceil \frac{n}{2} \right\rceil$ smaller elements to build a max heap. We use the remaining $\left\lfloor \frac{n}{2} \right\rfloor$ elements to build a min heap. The median will be at the root of max heap (We

assume the case of even n , median is $\frac{n}{2}th$ element when sorted in increasing order (5 pts).

Part (a) grading break down:

- Putting the smaller half elements in max heap and the other half in min heap (4 pts).
- Mention where the median will be at (1 pts).

b) For a new element x , we compare x with current median (root of max heap). If $x < median$ we insert x into max heap. Otherwise we insert x into min heap. If $size(maxHeap) > size(minHeap) + 1$, then we ExtractMax() on max heap and insert the extracted value into the min heap. Also if $size(minHeap) > size(maxHeap)$ we ExtractMin() on min heap and insert the extracted value in max heap. (6 pts).

Part (b) grading break down:

- Correctly identifying which heap to insert the value into (2 pts).
- Maintaining the size of the heaps correctly (4 pts).
- If you only got the idea that you have to maintain the size of the heaps equal but don't do it correctly (1 pt).

c) ExtractMax() on max heap. If after extraction $size(maxHeap) < size(minHeap)$ then we ExtractMin() on min heap and insert the extracted value in max heap (5 pts).

Part (c) grading break down:

- Correctly identifying which root to extract and retaining heap property (1 pt).
- Maintaining the size of heaps correctly (4 pts).
- If you only got the idea that you have to maintain the size of the heaps equal but don't do it correctly (1 pt).

CS570
Analysis of Algorithms
Summer 2008
Exam I

Name: _____

Student ID: _____

_____ 4:00 - 5:40 Section

_____ 6:00 – 7:40 Section

	Maximum	Received
Problem 1	20	
Problem 2	20	
Problem 3	10	
Problem 4	15	
Problem 5	10	
Problem 6	10	
Problem 7	15	
Total	100	

2 hr exam
Close book and notes

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[**TRUE/FALSE**]

There exist a perfect matching and a corresponding preference list such that every man is part of an instability, and every woman is part of an instability.

FALSE

[**TRUE/FALSE**]

A greedy algorithm always returns the optimal solution.

FALSE

[**TRUE/FALSE**]

The function $100n + 3$ is $O(n^2)$

FALSE

[**TRUE/FALSE**]

You are given n elements to insert into an empty heap. You could directly call heap insert n times. Or you could first sort the elements and then call heap insert n times. In either case, the asymptotic time complexity is the same.

TRUE

[**TRUE/FALSE**]

If a problem can be solved correctly using the greedy strategy, there will only be one greedy choice (e.g. "choose the object with highest value to weight ratio") for that problem that leads to the optimal solution.

FALSE

[**TRUE/FALSE**]

The Depth First Search algorithm can not be used to test whether a directed graph is strongly-connected.

FALSE

[**TRUE/FALSE**]

Consider an undirected graph $G=(V, E)$ with non-negative edge weights. Suppose all edge weights are different. Then the edge of maximum weight can be in the minimum spanning tree.

TRUE

[**TRUE/FALSE**]

Consider a perfect matching S where Mike is matched to Susan. Suppose Mike prefers Winona to Rachel. Then the pair (Mike, Rachel) can never be an instability with respect to S .

FALSE

[TRUE/FALSE]

Consider an undirected graph $G=(V, E)$. Suppose all edge weights are different. Then the shortest path from A to B must be unique.

FALSE

[TRUE/FALSE]

The number of elements in a heap must always be an integer power of 2.

FALSE

2) 20 pts

Given two graphs G and G' that have the same sets of vertices V and edges E , however different weight functions (W and W' respectively) on their edges. Suppose for each graph the weights on the edges are distinct and satisfy the following relation: $W'(e)=W(e)^2$ (Note: all edge weights in G and G' are positive integers) for every edge e of E . Decide whether each of the following statements is true. Give either a short proof or a counter example.

a) The minimum spanning tree of G is the same as the minimum spanning tree of G' .

Solution

This statement is true

Now, since the edges costs are all distinct, the order of the sorted lists of the edges will be the same in G and G' . This is because if $a > b$, then $a^2 > b^2$.

Now, if we run, Prim's algorithm, it will consider the edges in the same order in the case of both G and G' . Therefore, since the structure of G and G' are same and also the ordering of the edges (with respect to edge cost) is the same, the same MST will be produced.

b) For a pair of vertices a and b in V , a shortest path between them in G is also a shortest path in G' .

Solution

This statement is false. Hint : Pythagoras's theorem

3) 10 pts

Prove or give a counterexample: An array that is sorted in ascending order is a min-heap.

Solution

Let's assume that the statement is false. Suppose H is a binary tree for the array sorted in ascending order. By the statement, for every element v at a node i in H ,

1. the element w at i 's parent satisfies $\text{key}(w) \leq \text{key}(v)$.
2. the element w at node i and its right sibling w' satisfies $\text{key}(w) \leq \text{key}(w')$.

Min-heap property: for every element v at a node i and the element w at i 's parent satisfies $\text{key}(w) \leq \text{key}(v)$.

H satisfies the min-heap property. Contradiction. Thus, the statement is true.

4) 15 pts

Let $G = (V, E)$ be an undirected graph with maximum degree d . A coloring of G is an assignment to each vertex of G of a "color" such that adjacent vertices have distinct colors. Consider the greedy algorithm that colors as many vertices as possible with color " j " before moving to color " $j+1$."

Prove or give a counterexample: This greedy algorithm never requires more than $d+1$ colors to color G .

Solution:

Proof: Let $\chi(G)$ denotes number of colors this greedy algorithm requires to color G .

Prove by contradiction.

Assume there exists a graph G' with maximum degree d such that $\chi(G') \geq d+2$. Let v be the first vertex in G' colored with color " $d+2$ " by the algorithm. Hence all vertices adjacent to v must have used up color " 1 " through " $d+1$ " (otherwise by the algorithm color " $d+2$ " would not be used), which means v has at least $d+1$ adjacent vertices, that is, the degree of v is at least $d+1$. But the maximum degree in G' is d . Contradiction.

Therefore, this greedy algorithm never requires more than $d+1$ colors to color G .

5) 10 pts

You and your eight-year-old nephew Shrek decide to play a simple card game. At the

beginning of the game, the cards are dealt face up in a long row. Each card is worth a different number of points. After all the cards are dealt, you and Shrek take turns removing either the leftmost or rightmost card from the row, until all the cards are gone. At each turn, you can decide which of the two cards to take. The winner of the game is the player that has collected the most points when the game ends.

Having never taken an algorithms class, Shrek follows the obvious greedy strategy—when it's his turn, Shrek always takes the card with the higher point value. Your task is to find a strategy that will beat Shrek whenever possible. (It might seem mean to beat up on a little kid like this, but Shrek absolutely hates it when grown-ups let him win.)

Prove that you should not also use the greedy strategy. That is, show that there is a game that you can win, but only if you do not follow the same greedy strategy as Shrek.

Solution

Let S be Shrek's total point and S' be my total point at i th turn. Let $p_k, p_{k+1}, p_{k+2}, p_{k+3}, p_{k+4}, p_{k+5}$ be the sums of each pair of cards that remained, where $k \geq 1$ and $p_{k+2} > p_{k+3} > p_{k+4} > p_{k+1} > p_{k+2}$. By the greedy strategy, the order that Shrek picks the pair of cards would be $p_{k+4}, p_{k+2}, p_{k+1}$ and my order would be p_{k+3} then p_k . The total point at the final turn is that $S + p_{k+4} + p_{k+2} + p_{k+1} > S' + p_{k+3} + p_k$. If I don't use the greedy strategy, the result at the final turn would be $S + p_{k+4} + p_{k+3} + p_{k+1} < S' + p_k + p_{k+2}$. Contradiction. Thus, the greedy strategy is not an optimal solution.

6) 10 pts

Arrange the following functions in increasing order of asymptotic complexity. If $f(n) = \Theta(g(n))$ then put $f = g$. Else, if $f(n) = O(g(n))$, put $f < g$.

$4n^2, \log_2(n), 20n, 2, \log_3(n), n^n, 3^n, n \log(n), 2n, 2^{n+1}, \log(n!)$

Solution: $2 < \log_2(n) = \log_3(n) < 2n = 20n < n \log(n) = \log(n!) < 4n^2 < 2^{n+1} < 3^n < n^n$

7) 15 pts

Prove or give a counterexample: Let G be an undirected, connected, bipartite, weighted graph. If the weight of each edge in G is $+1$, and for every pair of vertices (u, v) in G there is exactly one shortest path, then G is a tree.

Solution

The statement is true.

It is same as the following statement:

G has loop(s) $\Rightarrow$ there exists at least one pair of vertices (u, v) in G with more than one shortest path.

G is a bipartite graph $\Rightarrow$ it can not contain an odd cycle $\Rightarrow$ all the cycle(s) have even number of nodes $\Rightarrow$ all the cycle(s) have length $2n$ (since edge weight is 1)

Take a smallest loop with $2m$ nodes and length $2m$ in G , then any pair of nodes (u,v) that are farthest away have two paths with equal length m . Also, since the loop is smallest, there doesn't not exist any path $< m$ between (u,v) .

Proved.

CS570
Analysis of Algorithms
Summer 2009
Exam I

Name: _____
Student ID: _____

	Maximum	Received
Problem 1	20	
Problem 2	10	
Problem 3	10	
Problem 4	20	
Problem 5	15	
Problem 6	10	
Problem 7	15	
Total	100	

2 hr exam

Close book and notes

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[**TRUE/FALSE**] **T**

An array with following sequence of terms [20, 15, 18, 7, 9, 5, 12, 3, 6, 2] is a max-heap.

[**TRUE/FALSE**] **T**

Complexity of the following shortest path algorithms are the same:

- Find shortest path between S and T
- Find shortest path between S and all other points in the graph

[**TRUE/FALSE**] **T**

In an undirected weighted graph with distinct edge weights, both the lightest and the second lightest edge are in the MST.

[**TRUE/FALSE**] **F**

Dijkstra's algorithm works correctly on graphs with negative-cost edges, as long as there are no negative-cost cycles in the graph.

[**TRUE/FALSE**] **T**

Not all recurrence relations can be solved by Master theorem.

[**TRUE/FALSE**] **F**

Mergesort does not need any additional memory space other than that held by the array being sorted.

[**TRUE/FALSE**] **T**

An algorithm with a complexity of $O(n^2)$ could run faster than one with complexity of $O(n)$ for a given problem.

[**TRUE/FALSE**] **F**

There are at least 2 distinct solutions to the stable matching problem--one that is preferred by men and one that is preferred by women.

[**TRUE/FALSE**] **F**

A divide and conquer algorithm has a minimum complexity of $O(n \log n)$ since the height of the recursion tree is always $O(\log n)$.

[**TRUE/FALSE**] **T**

Stable matching algorithm presented in class is based on the greedy technique.

2) 10 pts

a) Arrange the following in the increasing order of asymptotic growth. Identify any ties.

$$\lg n^{10}, 3^n, \lg n^{2n}, 3n^2, \lg n^{\lg n}, 10^{\lg n}, n^{\lg n}, n \lg n$$

If $\lg$ is $\log_2$ (convention),

$$\lg n^{10} < \lg n^{2n} < \lg n^{2n} = n \lg n < 3n^2 < 10^{\lg n} < n^{\lg n} < 3^n$$

If $\lg$ is $\log_{10}$ (from ISO specification)

$$\lg n^{10} < \lg n^{2n} < 10^{\lg n} < \lg n^{2n} = n \lg n < 3n^2 < n^{\lg n} < 3^n$$

b) Analyze the complexity of the following loops:

```
i- x = 0
  for i=1 to n
    x = x + lg n
  end for
```

$O(n)$

```
ii- x=0
   for i=1 to n
     for j=1 to lg n
       x = x * lg n
     endfor
   endfor
```

$O(n \log n)$

```
iii- x = 0
     k = "some constant"
     for i=1 to max (n, k)
       x = x + lg n
     end for
```

$O(n)$

```
iv- x=0
    k = "some constant"
    for i=1 to min(n, k)
      for j=1 to lg n
        x = x * lg n
      endfor
    endfor
```

$O(\log n)$

3) 10 pts

a) For each of the following recurrences, give an expression for the runtime $T(n)$ if the recurrence can be solved with the Master Theorem. Otherwise, indicate that the Master Theorem does not apply.

$$T(n) = T(n/2) + 2^n$$

$$\theta(2^n)$$

$$T(n) = 16T(n/4) + n$$

$$\theta(n^2)$$

$$T(n) = 2T(n/2) + n \log n$$

More general case 2

If $F(n) = \theta((n^{\log b(a)}) * (\log n)^k)$ with $k \geq 0$, then $T(n) = \theta((n^{\log b(a)}) * (\log n)^{(k+1)})$

$$\theta(n(\log n)^2) \text{ from case 2}$$

$$T(n) = 2T(n/2) + \log n$$

$$\theta(n)$$

$$T(n) = 64T(n/8) - n^2 \log n$$

Does not apply ($f(n)$ is not positive)

4) 20 pts

You work for a small manufacturing company and have recently been placed in charge of shipping items from the factory, where they are produced, to the warehouse, where they are stored. Every day the factory produces n items which we number from 1 to n in the order that they arrive at the loading dock to be shipped out. As the items arrive at the loading dock over the course of the day they must be packaged up into boxes and shipped out. Items are boxed up in contiguous groups according to their arrival order; for example, items 1... 6 might be placed in the first box, items 7...10 in the second, and 11... 42 in the third.

Items have two attributes, *value* and *weight*, and you know in advance the values $v_1 \dots v_n$ and weights $w_1 \dots w_n$ of the n items. There are two types of shipping options available to you:

Limited-Value Boxes: One of your shipping companies offers insurance on boxes and hence requires that any box shipped through them must contain no more than V units of value. Therefore, if you pack items into such a “limited-value” box, you can place as much weight in the box as you like, as long as the total value in the box is at most V .

Limited-Weight Boxes: Another of your shipping companies lacks the machinery to lift heavy boxes, and hence requires that any box shipped through them must contain no more than W units of weight. Therefore, if you pack items into such a “limited-weight” box, you can place as much value in the box as you like, as long as the total weight inside the box is at most W .

Please assume that every individual item has a value at most V and a weight at most W . You may choose different shipping options for different boxes. Your job is to determine the optimal way to partition the sequence of items into boxes with specified shipping options, so that shipping costs are minimized.

Suppose limited-value and limited-weight boxes each cost \$1 to ship. Describe an $O(n)$ greedy algorithm that can determine a minimum-cost set of boxes to use for shipping the n items. Justify why your algorithm produces an optimal solution.

We use a greedy algorithm that always attempts to pack the largest possible prefix of the remaining items that still fits into some box, either limited-value or limited weight. The algorithm scans over the items in sequence, maintaining a running count of the total value and total weight of the items encountered thus far. As long as the running value count is at most V or the running weight count is at most W , the items encountered thus far can be successfully packed into some type of box. Otherwise, if we reach a item j whose value and weight would cause our counts to exceed V and W , then prior to processing item j we first package up the items scanned thus far (up to item $j-1$) into an appropriate box and zero out both counters. Since the algorithm spends only a constant amount of work on each item, its running time is $O(n)$.

Why does the greedy algorithm generate an optimal solution (minimizing the total number of boxes)? Suppose that it did not, and that there exists an optimal solution different from the greedy solution that uses fewer boxes. Consider, among all optimal solutions, one which agrees with the greedy solution in a maximal prefix of its boxes. Let us now examine the sequence of boxes produced by both solutions, and consider the first box where the greedy and optimal solutions differ. The greedy box includes items $i \dots j$ and the optimal box includes items $i \dots k$, where $k < j$ (since the greedy algorithm always places the maximum possible number of items into a box). In the optimal solution, let us now remove items $k+1 \dots j$ from the boxes in which they currently reside and place them in the box we are considering, so now it contains the same set of items as the corresponding greedy box. In so doing, we clearly still have a feasible packing of items into boxes and since the number of boxes has not changed, this must still be an optimal solution; however, it now agrees with the greedy solution in one more box, contradicting the fact that we started with an optimal solution agreeing maximally with the greedy solution.

5) 15 pts

For two unsorted arrays x and y , each of size n , give a divide and conquer algorithm that runs in $O(n)$ time and computes the maximum sum M , as given below:

$$M = \text{Max} (x_i + x_{i+1} - y_i); \quad i=1 \text{ to } n-1$$

Divide: divide problem into 2 equal size subproblems at each step.

Conquer: solve the subproblems recursively until the subproblem become trivial to solve. In this case problem size of 2 is trivial. In that case the solution is the sum of the 2 numbers.

Combine: We need to merge the solutions to the two subproblems $S1$ and $S2$. At each step we need to evaluate the following 3 cases:

Case 1: Max is in $S1$ (subproblem to the left)

Case 2: Max is in $S2$ (subproblem to the right)

Case 3: Max is on the border of $S1$ and $S2$

In case 3, you need to calculate $X(\text{the last element of } S1) + X(\text{the first element of } S2) - Y(\text{the last element of } S1)$

6) 10 pts

Suppose we are given an instance of the Shortest Path problem with source vertex s on a directed graph G . Assume that all edges costs are positive and distinct. Let P be a minimum cost path from s to t . Now suppose that we replace each edge cost c_e by its square, c_e^2 , thereby creating a new instance of the problem with the same graph but different costs.

Prove or disprove: P still a minimum-cost $s - t$ path for this new instance.

Let G have edges (s,v) , (v,t) , and (s,t) , when the first two of these edges have cost 3 and the third has cost 5. Then the shortest path is the single edge (s,t) , but after squaring the costs the shortest path would go through v .

7) 15 pts

We are given two arrays of integers $A[1..n]$ and $B[1..n]$, and a number X . Design an algorithm which decides whether there exist $i, j \in \{1, \dots, n\}$ such that $A[i] + B[j] = X$. Your algorithm should run in time $O(n \log n)$.

Sort array B

For int $i = 0$ to n

 Temp = $X - a[i]$

 Do binary search to find an element whose value is equal to $X - A[i]$ in array B

 If it finds return true

End of For